

**EDGE AI WEARABLE HEALTH MONITORING SYSTEM WITH REAL-TIME
ANOMALY DETECTION AND DECISION SUPPORT**

By

PAUL PHILIP KADALA

(SFE/23U/3660)

July, 2026.

**EDGE AI WEARABLE HEALTH MONITORING SYSTEM WITH REAL-TIME
ANOMALY DETECTION AND DECISION SUPPORT**

BY

**PAUL PHILIP KADALA
(SFE/23U/3660)**

**Project submitted to the Department of Software Engineering, Faculty of Computing,
Nigerian Army University, Biu, in Partial Fulfilment of the Requirements for the
Award of Bachelor of Science Degree in Software Engineering**

July, 2026

DECLARATION

I declare that the work described in this project is original, and has not been previously submitted to any University or similar institutions for the award of any degree or certificate.

Name of Candidate:

Matric Number:

Signature:

Date:

CERTIFICATION

We the undersigned, hereby certify that this project presented by **PAUL PHILIP KADALA (Matric No. SFE/23U/3660)** be accepted as fulfilling part of requirements for the award of degree of Bachelor of Science in Software Engineering.

Title: **EDGE AI WEARABLE HEALTH MONITORING SYSTEM WITH REAL-TIME ANOMALY DETECTION AND DECISION SUPPORT**

Dr. Muhammad Aliyu

Supervisor

.....

Signature/Date

Mr. Adamu Liman Muhammed

Project Coordinator

.....

Signature/Date

Dr. Ali Garba Jakwa

Head of Department

.....

Signature/Date

DEDICATION

This project is dedicated, first and foremost, to Almighty **God**, whose grace sustained me through every stage of this work and without whom none of this would have been possible. To my **father** and my **mother**, whose sacrifices, prayers, and unwavering support gave me the foundation to pursue this degree, I owe more than words can express. To my sister, thank you for the encouragement and the quiet strength you brought on the difficult days. To my supervisor, **Dr. Muhammad Aliyu**, thank you for your guidance, patience, and the standard of excellence you held me to throughout this project. To the Nigerian Army University, Biu, this work is a product of the opportunities this institution gave me. And to my friends who stood by me through the long nights and the hard moments, this is as much yours as it is mine.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project supervisor, **Dr. Muhammad Aliyu**, for his guidance and wealth of experience, which proved central to shaping the direction and quality of this research from start to finish. His feedback, depth of knowledge, and dedication to seeing this work meet the right standard kept this project on the right path.

I also wish to acknowledge the Head of Department, **Dr. Ali Garba Jakwa**, for his leadership within the Department of Software Engineering, and all the lecturers of the department whose teachings over the course of my studies laid the foundation upon which this work was built.

To my **parents**, I owe a debt no words can fully settle. You funded this journey, prayed over it without ceasing, and never once allowed me to consider giving up. I hope this work is a worthy return on everything you invested. To my **sister**, thank you for being a constant source of encouragement through the most demanding seasons of this program.

To my friends and fellow students, thank you for the company, the conversations, the late nights, and the solidarity that made a difficult journey feel less lonely.

Finally and above all, I give glory to **God Almighty**, whose grace was sufficient at every point where my own strength fell short. This project stands as a testament to His faithfulness.

TABLE OF CONTENTS

COVER PAGE.....	i
TITLE PAGE.....	ii
DECLARATION	iii
CERTIFICATION	iv
DEDICATION	v
ACKNOWLEDGEMENT	vi
TABLE OF CONTENTS.....	vii
LIST OF TABLES	x
LIST OF FIGURES	xi
ABSTRACT.....	xii
CHAPTER ONE	1
INTRODUCTION	1
1.0 Background of the Study	1
1.1 Statement of the Problem.....	2
1.2 Aim and Objectives of the Study	3
1.3 Scope and Limitation of the Study.....	4
1.4 Significance of the Study	4
1.5 Definition of Terms.....	5
CHAPTER TWO	8
LITERATURE REVIEW	8
2.0 Introduction.....	8
2.1 Theoretical Framework.....	8
2.2 Conceptual Framework.....	9
2.3 Wearable Health Monitoring Systems: Evolution and Limitations.....	10
2.4 Mobile Edge Computing and Machine Learning Integration in Healthcare.....	11
2.5 Anomaly Detection in Physiological Signals	11
2.6 The WESAD Dataset and Its Relevance to This Project.....	12
2.7 Machine Learning	13
2.7.1 Type	13
2.7.2 Benefit.....	14
2.7.3 Challenges.....	14
2.7.4 Machine Learning Deployment on Mobile Edge Nodes	14
2.8 Privacy, Security, and Data Sovereignty	15

2.9 Multi-Modal Sensing and Sensor Fusion.....	15
2.10 Literature Mapping: Recent Advancements in Mobile Edge AI Health Monitoring .	16
2.11 Research Gap	19
CHAPTER THREE	21
SYSTEM ANALYSIS AND DESIGN.....	21
3.0 Introduction.....	21
3.1 Methodology	21
3.2 Research Design and Data Strategy	22
3.2.1 Nature of the Research.....	22
3.2.2 The Research Artifact	23
3.2.3 Training Data Strategy	23
3.2.4 Evaluation Strategy	23
3.3 Machine Learning Pipeline	24
3.3.1 Dataset Acquisition.....	24
3.3.2 Data Cleaning.....	24
3.3.3 Feature Extraction	25
3.3.4 Model Training	25
3.3.5 Model Validation	26
3.3.6 Model Deployment Using TensorFlow Lite	26
3.4 Analysis of the Existing System	27
3.4.1 Description of Existing Systems.....	27
3.4.2 Shortcomings of the Existing System.....	27
3.5 System Design	28
3.5.1 System Overview	28
3.5.2 Functional Requirements	29
3.5.3 Non-Functional Requirements	30
3.5.4 Hardware Component Design.....	31
3.5.5 Mobile Application Design.....	32
3.6 System Architecture and Models	33
3.6.1 Three-Layer System Architecture.....	33
3.6.2 Use Case Model	33
3.6.3 Class Design.....	34
3.7 Design Flowcharts and Algorithms	35
3.7.1 Main System Operation Flowchart	35

3.7.2 BLE Data Transmission Protocol	37
3.8 Advantages of the System.....	37
CHAPTER FOUR.....	39
SYSTEM IMPLEMENTATION	39
4.0 Introduction.....	39
4.1 System Implementation	39
4.1.1 Development Environment and Tools	39
4.1.2 Wearable Hardware	40
4.1.3 Firmware and BLE Interface	42
4.1.4 Flutter Mobile Application	42
4.1.5 Machine Learning and TFLite Deployment	50
4.2 RESULTS	50
4.3 DISCUSSION	55
4.4 System Documentation and User Manual.....	56
4.4.1 Hardware Setup and Power-On	56
4.4.2 Mobile Application Installation	57
4.4.3 Pairing the Wearable Device with the Application.....	57
4.4.4 Using the Live Dashboard	57
4.4.5 Understanding and Responding to Alerts	57
4.4.6 Using the AI Agent	58
4.4.7 Reviewing Historical Readings.....	58
CHAPTER FIVE	59
SUMMARY, CONCLUSION AND RECOMMENDATION.....	59
5.0 Summary	59
5.1 Conclusion	59
5.2 Limitation of the Study	60
5.3 Recommendations.....	60
5.4 Contribution to Knowledge.....	60
REFERENCES	62
APPENDIX.....	65
SOURCE CODE LISTING	65

LIST OF TABLES

2.1 Literature Mapping.....	16
3.1 Functional Requirements.....	29
3.2 Non-Functional Requirements.....	30
3.3 Comparative Analysis of Existing System Against the System.....	38
4.1 Development Tools.....	40
4.2 ESP32 Pin Connections.....	41
4.3 BiLSTM Model Performance	54
4.4 TFLite Model Conversion Summary.....	55
4.5 System Test Cases and Results.....	55

LIST OF FIGURES

3.1 DSRM diagram.....	22
3.2 Circuit diagram.....	32
3.3 UML Use Case Diagram	34
3.4 UML Class Diagram.....	35
3.5 System Operation Flowchart.....	37
4.1 development.....	42
4.2 wrist-worn device.....	43
4.3 Mobile app screen showing BLE device scanning	44
4.4 Mobile app dashboard.....	46
4.5 Mobile application anomaly detection.....	48
4.6 Mobile app AI agent conversation screen.....	49
4.7 Mobile app history charts	50
4.8 Overall WESAD Label Distribution.....	52
4.9 Full Session Signal Plot for Subject S2.....	53
4.10 Processed Normal and Anomaly.....	54

ABSTRACT

Wearable health monitoring devices currently in use remain heavily dependent on continuous cloud connectivity for meaningful health analysis, a dependence that introduces critical delays during medical emergencies, fails outright in regions with poor network coverage, and relies on generalized, population-based models that overlook the physiological differences between individuals. This study addressed these limitations by designing and implementing a hybrid mobile edge AI wearable health monitoring system capable of personalized, real-time anomaly detection directly on a user's smartphone without dependence on internet connectivity. The Design Science Research methodology was adopted, involving the assembly of an ESP32-based wrist-worn sensing device with sensors, Bluetooth Low Energy communication with a Flutter mobile application, and the training of a Bidirectional Long Short-Term Memory (BiLSTM) anomaly detection model on the WESAD dataset using leave-one-subject-out cross-validation. TensorFlow and TensorFlow Lite were used to build, train, and quantize the model, with Google Colab as the cloud training environment, while Arduino IDE and Visual Studio Code were used for implementation, development, and testing. Results showed that the trained model achieved an accuracy of 91.8 percent, a precision of 89.6 percent, a recall of 87.9 percent, and an F1 score of 88.7 percent on held-out test data, while the quantized on-device model recorded an average inference latency of 64 milliseconds, and live system testing confirmed reliable Bluetooth Low Energy communication, real-time dashboard updates, and full offline anomaly detection and alerting functionality. These results demonstrate that a personalized health-monitoring system can be built using low-cost, open-source tools without sacrificing performance in offline operation. The study therefore recommends that developers and healthcare technology providers serving resource-constrained and low-connectivity regions, particularly in Nigeria, adopt mobile edge computing and on-device inference approaches as a practical pathway toward more accessible and privacy-preserving health monitoring for underserved communities.

CHAPTER ONE

INTRODUCTION

1.0 Background of the Study

The integration of computing technology into healthcare has grown significantly over the past two decades. Wearable health monitoring devices are a major part of this growth. People wear these devices during daily activities to track physiological metrics like heart rate, body temperature, movement, and blood oxygen levels without invasive procedures. Consumer products like the Apple Watch and Fitbit show that clinically useful data can be collected outside of hospitals. This makes health monitoring accessible to people who might not regularly visit a clinic. Wearables have quickly grown from fitness gadgets to tools recommended by doctors and used in medical research (Bent et al., 2020).

As wearables proved commercially successful, developers invested heavily in turning them from simple display units into genuinely analytical tools. Where early devices did little more than show raw sensor numbers, current systems apply machine learning to continuous physiological streams to classify sleep stages, track stress, and flag irregular heart rhythms as standard features (Pereira et al., 2024). This growth in analytical capability, however, has come with a structural cost: the intelligence behind these features rarely lives on the device itself but on remote cloud servers that demand an uninterrupted internet connection to function. That single dependency generates three compounding problems. Health analysis simply stops working the moment connectivity drops, shutting out users in rural or developing regions, Nigeria included, where network infrastructure is unreliable, and the people who stand to benefit most are left without access. Even when the connection holds, the round-trip to a remote server introduces a delay that, though negligible in everyday use, can be dangerous when a system must detect an acute medical anomaly in real time. And transmitting raw physiological data to third-party servers exposes users to serious privacy and security risks (Cruz Castaneda and Bertemes Filho, 2024).

A second weakness lies in how these systems are trained to recognize anomalies at all. Because anomaly detection algorithms are typically built from massive, aggregated datasets meant to represent population norms, they are poorly suited to the individual wearing the device;

physiological signals vary too much between people for a generalized model to reliably tell what is dangerous for one user apart from what is simply normal for another, producing both missed anomalies and frequent false alarms (Ghorbani et al., 2024). Mobile edge AI addresses both failures directly by relocating intelligence from the cloud to the user's smartphone. Running anomaly detection locally on a nearby device rather than a distant server means the system can operate without internet access, avoid communication delays entirely, and keep sensitive physiological data confined to the user's own phone (Alajlan and Ibrahim, 2022).

This project is a hybrid wearable health-monitoring system designed to overcome these exact limitations. The design combines a custom wrist-worn sensor unit built on the ESP32 microcontroller with a Flutter-based mobile application. The wearable unit collects the data and transmits it via Bluetooth to the mobile app. The mobile app serves as the edge node that houses the machine learning model. The system operates fully offline for continuous monitoring and anomaly detection. It can also connect to the cloud when the internet is available for more complex interactions. This setup guarantees a reliable and private monitoring tool that works anywhere.

1.1 Statement of the Problem

Current wearable health monitoring systems suffer from two critical structural flaws that prevent them from functioning as reliable medical tools. The first is an absolute dependence on cloud connectivity. Because existing systems route raw sensor data to remote servers for analysis, their intelligent features fail entirely in areas with poor network coverage, which is a widespread condition across many parts of Nigeria and other resource-limited environments. Pereira et al. (2024) identified network dependency as a critical flaw in a systematic mapping of 171 wearable health studies, confirming that very few existing systems deliver meaningful feedback without a connection. The second flaw is the use of generalized, population-based models that treat every user identically. Ghorbani et al. (2024) showed that such models score only marginally above random guessing when evaluated against individual physiological patterns, while personalized models achieve near-perfect accuracy on the same cases. Beyond these two problems, routing raw physiological data to third-party servers introduces serious privacy risks, as Cruz Castaneda and Bertemes Filho (2024) identified in their work on edge-IoT health architectures.

These overlapping weaknesses reveal the need for a fundamentally different approach, one in which intelligent health analysis runs locally on the user's device, adapts to the individual's own physiological baseline, and operates reliably without any internet connection. This project addresses that need by designing and implementing an offline-first, privacy-preserving wearable health monitoring system that performs real-time anomaly detection directly on the user's smartphone using mobile edge computing and a personalized BiLSTM model trained on the WESAD dataset.

1.2 Aim and Objectives of the Study

The aim of this project is to design and implement a hybrid mobile edge AI wearable health monitoring system capable of performing personalized, real-time anomaly detection locally on the user's mobile device.

The specific objectives of this study are to:

- I. Develop a wrist-worn device using the ESP32 (Espressif Systems 32-bit) microcontroller, integrating photoplethysmography (PPG), temperature, and accelerometer sensors for real-time physiological monitoring.
- II. Configure Bluetooth Low Energy communication to transmit data to a Flutter mobile app for visualization, storage, and alerting without requiring internet access.
- III. Deploy a Bidirectional Long Short-Term Memory (BiLSTM) anomaly detection model, trained on the Wearable Stress and Affect Detection (WESAD) dataset and converted to TensorFlow Lite, within the mobile app to identify physiological anomalies from real-time sensor data.
- IV. Evaluate the system's performance regarding accuracy, latency, reliability, and effectiveness across both offline and online modes.

1.3 Scope and Limitation of the Study

This study focuses on engineering a functional prototype to validate the mobile edge computing architecture. The monitoring scope is strictly limited to the chosen sensor suite, covering heart rate and blood volume pulse derived from PPG, skin surface temperature, and three-axis physical movement. The project does not cover parameters that require specialized medical hardware, such as continuous blood pressure, blood glucose, or full electrocardiography.

The hardware consists of a printed veroboard prototype using the ESP32 development board and commercially available sensors. The goal is to prove the system's communication architecture and local AI inference capabilities rather than to manufacture a miniaturized consumer watch. The mobile application is built using Flutter and optimized primarily for Android. While the cross-platform codebase can theoretically run on iOS, formal testing on Apple devices is outside the current scope.

For the AI components, the anomaly detection model is trained on the WESAD (Wearable Stress and Affect Detection) publicly available dataset using only the wrist device sensor streams, which match the sensors used in this project. The intelligence framework uses localized logic for offline support and an external API for online interactions. System evaluation relies on quantitative performance metrics gathered through live hardware testing.

1.4 Significance of the Study

This research offers valuable contributions to academic literature, practical engineering, and healthcare accessibility. Academically, the project bridges embedded systems, mobile software engineering, and machine learning to create a fully integrated mobile edge computing architecture. Previous reviews, such as Pereira et al. (2024), highlight that most current studies focus on isolated hardware tests or on algorithm accuracy without demonstrating a complete, end-to-end system. This project provides a comprehensive reference architecture that spans the hardware, the mobile application, and the AI inference.

In practice, the system addresses a major healthcare equity issue. In regions like Nigeria, where reliable internet access cannot be guaranteed, the ability to run advanced health monitoring offline significantly increases the technology's usefulness. The system works in

environments where traditional consumer wearables fail, making it highly suitable for resource-constrained areas.

The most meaningful contribution of this work is twofold. First, the system builds a personalized physiological baseline from each user's historical readings rather than relying on population averages, a practice several studies have shown to be critical for reducing false alarms. Second, the system embeds a dual-mode conversational AI agent that can respond to health queries entirely offline, a feature currently absent from commercial wearable devices. Together, these two additions address major limitations that existing systems have not resolved. Furthermore, the architecture inherently protects user privacy. By processing raw physiological data locally on the smartphone, the system complies with modern data protection regulations and addresses growing public concern about the exploitation of health data. It demonstrates that high-quality monitoring does not require sacrificing data sovereignty.

1.5 Definition of Terms

The following terms are defined as they are used within the context of this study:

- I. **Mobile Edge AI:** The practice of running artificial intelligence inference on a nearby device such as a smartphone, rather than on centralized cloud servers. In this study, mobile edge AI refers to deploying the anomaly detection model entirely within the Flutter mobile application on the user's smartphone, which is the primary intelligence node of the system.
- II. **TinyML (Tiny Machine Learning):** A specialized subfield of machine learning focused on compressing models small enough to run on ultra-low-power microcontrollers with very limited memory. This project deliberately does not run inference on the ESP32 microcontroller. Instead, inference runs on the more capable smartphone, which is classified as mobile edge AI. TinyML is referenced in this study only to distinguish it from the approach adopted here.
- III. **TensorFlow Lite (TFLite):** An open-source framework developed by Google for running machine learning models on mobile and embedded devices. A model trained in TensorFlow is converted into a compressed .tflite file through a process called

quantization, which reduces the model's numerical precision to make it smaller and faster without significantly affecting its accuracy. The .tflite file is then packaged inside the Flutter app and executed locally on the smartphone, enabling the system to run anomaly detection with no internet connection.

- IV. **Wearable Device:** An electronic device worn on or close to the body that continuously collects physiological data. In this study, it refers to the custom-built ESP32 hardware prototype designed for the wrist, which acts solely as a data acquisition and transmission node.
- V. **Photoplethysmography (PPG):** An optical sensing technique that measures changes in blood volume in the peripheral capillary beds. It is used in this project to derive cardiovascular metrics via the MAX30102 sensor mounted on the ESP32.
- VI. **Accelerometer:** An inertial sensor that measures the rate of change of velocity. In this project, the MPU6050 sensor captures movement to serve as a physiological indicator and to help filter motion artifacts from the PPG signal.
- VII. **Bluetooth Low Energy (BLE):** A short-range wireless communication protocol optimized for low power consumption. It is used here to stream physiological data from the ESP32 wearable directly to the Flutter application.
- VIII. **Anomaly Detection:** A machine learning technique used to identify data points that deviate significantly from a normal baseline. In this context, it means identifying real-time physiological readings within the mobile app that suggest a concerning health state.
- IX. **Flutter:** A cross-platform mobile development framework created by Google. It is used in this project to build the mobile application that acts as the local edge inference node.
- X. **ESP32:** A low-power microcontroller featuring integrated Wi-Fi and Bluetooth. It serves as the central processing unit of the wearable sensing device, responsible for gathering and sending sensor data.
- XI. **Decision Support System:** A non-diagnostic software module that provides actionable health alerts. Offline, it uses rule-based logic and the local AI model to flag deviations

from clinical norms. Online, it uses an external large language model to provide conversational context regarding the user's data.

- XII. **Dual-Modal Operation:** The design principle that allows the system to maintain full monitoring and alerting capabilities offline while utilizing cloud-connected AI services for enhanced insights when the internet is available.
- XIII. **On-device Inference:** The execution of a trained machine learning model directly within the mobile application on the user's smartphone. This ensures that the data collected by the wearable hardware is processed locally, with zero data transmission to external servers for core anomaly detection.
- XIV. **WESAD Dataset:** The Wearable Stress and Affect Detection dataset, a publicly available multimodal dataset recorded from 15 subjects wearing physiological sensors under controlled emotional conditions. In this study, only the wrist device sensor streams from WESAD, covering blood volume pulse, body temperature, and three-axis acceleration, are used to train the anomaly detection model, as these correspond directly to the sensors deployed on the ESP32 prototype.
- XV. **Design Science Research (DSR):** The research methodology adopted in this study. DSR is a systematic, artifact-oriented approach that generates knowledge by designing, building, and rigorously evaluating a solution artifact in response to a clearly identified practical problem. In this study, the artifact is the wearable health monitoring system itself (Peffer et al., 2007; Hevner, 2007).

CHAPTER TWO

LITERATURE REVIEW

2.0 Introduction

Designing a wearable health monitoring system that includes local artificial intelligence requires combining ideas from three main areas of research: wearable sensor technology, mobile edge computing, and biomedical signal processing. Reviewing the existing literature in these areas helps to understand current technological limits, justify the system's design choices, and clearly define the research gaps this project addresses.

2.1 Theoretical Framework

This project is grounded in two established theoretical foundations that together explain why the architecture is both necessary and technically sound.

The first is Edge Computing Theory. Edge computing is built on the principle that computation should occur as close to the data source as possible, rather than routing data to a distant central server. This theory argues that moving processing to the network edge reduces latency, preserves privacy, and removes dependency on continuous connectivity. For health monitoring, this translates directly: a system in which the analytical intelligence runs on the user's smartphone can respond to physiological changes in real time, without waiting for a round trip to a cloud server. The mobile edge AI approach in this project applies this principle by treating the smartphone as the primary intelligence node and the ESP32 wearable as a data acquisition device only.

The second theoretical foundation is Personalized Anomaly Detection Theory. Traditional anomaly detection relies on threshold values drawn from population-wide statistics, flagging any reading that falls outside what is typical for most people. However, physiological signals are inherently individual. What constitutes an elevated heart rate for one person may be completely normal for another. Personalized anomaly detection theory holds that a reliable health monitor must first learn what is normal for a specific individual, and then detect deviations from that personal baseline. Ghorbani et al. (2024) provided empirical evidence for this theory, showing that generalized models achieved detection accuracy barely above chance

at the individual level, whereas personalized models achieved near-perfect accuracy. This theoretical position directly informs the baseline computation engine at the heart of the system.

2.2 Conceptual Framework

The conceptual framework below describes how the key components of this project relate to each other and jointly produce the system's central outputs.

At the base level, the ESP32 wearable hardware continuously collects three physiological signals: blood volume pulse from the MAX30102 sensor, body temperature from the DS18B20 sensor, and three-axis motion from the MPU6050 accelerometer. These raw signals are transmitted over Bluetooth Low Energy to the Flutter mobile application on the user's smartphone.

Within the mobile application, the incoming data flows through two parallel processes. The first is the personalization layer, where readings are stored in a local SQLite database and used to continuously build and update the user's personal physiological baseline. This baseline represents what is normal for that specific individual. The second is the inference layer, where each incoming reading is normalized to the personal baseline and evaluated by the on-device TFLite anomaly detection model, which was pre-trained on the WESAD dataset's wrist-sensor streams.

When the model flags a reading as anomalous, the alert engine generates an immediate notification. The user can then interact with the conversational AI agent, which operates in one of two modes. In offline mode, the agent applies a rule-based logic tree to interpret the alert and respond to the user's questions. In online mode, when internet access is available, the agent consults an external large language model to provide richer guidance. At no point does raw physiological data leave the device.

This framework positions the smartphone as the unifying node that connects hardware-level sensing, machine-learning inference, personalized health monitoring, and conversational decision support into a single, integrated system.

2.3 Wearable Health Monitoring Systems: Evolution and Limitations

Wearable health devices have evolved significantly over the last ten years. Early commercial fitness bands primarily counted steps and tracked basic sleep patterns. However, the refinement of photoplethysmography technology allowed manufacturers to integrate reliable optical heart rate monitors into wrist-worn devices. Today, systems from companies like Apple, Garmin, and Fitbit can estimate blood oxygen saturation and detect irregular heart rhythms, which are functions that previously required clinical equipment. This progress, however, also reveals a central limitation: consumer wearables still struggle to deliver reliable health monitoring when accuracy, software design, and connectivity matter most.

Despite these advancements, researchers continue to question the accuracy of consumer-grade optical sensors in real-world conditions. Bent et al. (2020) conducted a rigorous assessment of heart rate accuracy across multiple wearable devices. They found that while skin tone did not significantly affect accuracy, physical activity led to a severe drop in reliability. Specifically, the absolute error in heart rate measurements was roughly 30 percent higher during physical activity than during periods of rest. This underscores a major engineering challenge: the physiological events that require the most accurate monitoring often occur during physical exertion, precisely when motion artifacts compromise sensor data.

Beyond hardware accuracy, the software architecture of current wearables presents a major limitation. The standard industry approach treats the wearable device and the user's smartphone merely as data-collection tools. The actual computational work, such as pattern recognition and anomaly detection, is sent to remote cloud servers. Pereira et al. (2024) systematically mapped 171 studies on machine learning and wearable healthcare devices. They found that most systems focus on isolated hardware metrics or algorithmic accuracy without offering a complete, standalone architecture. Furthermore, their analysis identified network dependency as a critical flaw, noting that very few systems provide meaningful intelligent feedback when an internet connection is unavailable.

This heavy reliance on cloud infrastructure introduces several practical problems. The round-trip data transmission between the device and the cloud introduces latency, delaying emergency health alerts. Sending sensitive health data over public internet networks introduces

severe privacy risks. Finally, requiring constant internet access excludes users in areas with poor network coverage, making these advanced health tools inequitable.

2.4 Mobile Edge Computing and Machine Learning Integration in Healthcare

Integrating mobile edge computing with machine learning offers a direct solution to the limitations of cloud-centric wearables. Mobile edge computing processes data locally on a nearby device like a smartphone, rather than sending it to a remote server. This approach drastically reduces latency, ensures privacy, and allows the system to function offline.

In their comprehensive review, Pereira et al. (2024) noted that while researchers have proven the feasibility of edge-based health analysis, the field suffers from fragmented development. Most studies evaluate either the hardware performance or the artificial intelligence model in isolation, failing to demonstrate how they work together in a practical deployment. This highlights the need for end-to-end systems that seamlessly connect the hardware sensors, the mobile application, and the machine learning model.

Cruz Castaneda and Bertemes Filho (2024) explored this concept by proposing a three-layer Edge-IoT architecture for managing chronic diseases. Their system distributed the computing workload across the wearable device, an edge node, and the cloud, showing that processing raw data locally greatly improved response times and data privacy. However, their implementation still required the cloud layer to perform the final model inference, meaning the system would fail without internet access. The project in this study takes this architecture a step further by executing the core anomaly detection entirely on the mobile device, eliminating the dependency on the cloud for critical alerts.

2.5 Anomaly Detection in Physiological Signals

Detecting anomalies in time-series health data is fundamentally different from basic classification tasks. Instead of matching data to predefined disease categories, anomaly detection identifies any reading that significantly deviates from a normal baseline. This is highly appropriate for health monitoring because medical emergencies can present themselves in unpredictable ways.

A major technical hurdle in detecting these anomalies is the natural biological difference between individuals. Ghorbani et al. (2024) directly addressed this issue by proving that generalized artificial intelligence models, which are trained on broad population data, perform poorly on an individual level. They utilized representation learning to create personalized baselines for users. In their testing, generalized models achieved an accuracy score of barely 0.5, similar to random guessing, while personalized models achieved scores approaching a perfect 1.0. This confirms that for a health monitor to be reliable and avoid constant false alarms, the anomaly detection model must be personalized to the specific user wearing the device.

To analyze these complex signals, researchers have increasingly turned to hybrid deep learning models. Chen et al. (2025) demonstrated that combining Residual Networks and Bidirectional Long Short-Term Memory networks provides excellent results for extracting spatiotemporal features from PPG data. The convolutional component extracts the physical shape of the heart pulse, while the recurrent component tracks the rhythm over time. Deploying a compressed version of this architecture on a mobile device allows the system to analyze both the instantaneous shape and the long-term rhythm of the user's pulse locally.

2.6 The WESAD Dataset and Its Relevance to This Project

The WESAD dataset, introduced by Schmidt et al. (2018), is a publicly available multimodal dataset designed specifically for wearable stress and affect detection. The dataset was recorded from 15 subjects who wore two devices simultaneously: a chest-worn RespiBAN and a wrist-worn Empatica E4. Each subject went through a controlled study protocol that lasted approximately two hours and included a neutral baseline period, an amusement phase triggered by watching humorous video clips, and a stress phase induced through the Trier Social Stress Test, which involves public speaking and mental arithmetic tasks under observation.

This project uses only the wrist device streams from WESAD, specifically the blood volume pulse signal sampled at 64 Hz, body temperature sampled at 4 Hz, and three-axis accelerometer data sampled at 32 Hz. These three signals correspond directly to the sensors mounted on the ESP32 wearable prototype: the MAX30102 for PPG, the DS18B20 for temperature, and the MPU6050 for acceleration. This alignment makes WESAD a suitable foundation for training

the anomaly detection model, since the signal types and placement are consistent with the hardware this project deployed.

For the purposes of this study, the classification task is framed as a binary problem. The baseline condition, which represents normal resting physiology, forms the normal class. The stress condition, which represents a state of acute physiological deviation, forms the anomaly class. The amusement condition is excluded from training to keep the problem focused on the clinically relevant distinction between normal and anomalous states. This binary framing is consistent with the system's operational goal of alerting users when their readings deviate meaningfully from their personal norm rather than categorizing the nature of that deviation.

The dataset's 15-subject size is a recognized limitation in the wider literature. To address this during the personalization phase, the system further adapts its detection thresholds to each individual user's own stored readings collected over time, building on the WESAD-trained model as a starting point. This approach is supported by Ghorbani et al. (2024), whose work demonstrated that models fine-tuned to individual physiological patterns significantly outperform those relying solely on population-level training data.

2.7 Machine Learning

The way a trained machine learning model is deployed, whether in the cloud, on a smartphone, or on a microcontroller, determines a health monitoring system's latency, privacy, and offline capability. The following subsections cover the types of deployment relevant to mobile edge systems, their benefits, their challenges, and how this study applies them.

2.7.1 Type

Machine learning deployment falls into three types. Cloud-based inference sends sensor data to a remote server for processing and remains the industry default despite requiring constant connectivity. On-device inference on a smartphone runs a compressed model, such as TensorFlow Lite, directly within a mobile app using the phone's own resources (Zhang et al., 2025). On-device inference on a microcontroller, or TinyML, runs a highly compressed model directly on a constrained embedded device such as the ESP32, without a smartphone or server involved (Alajlan & Ibrahim, 2022; Yahyati et al., 2025).

2.7.2 Benefit

Edge deployment removes the round-trip delay of cloud communication, allowing anomalies to be detected in real time (Ali et al., 2025), and continues functioning without internet access. Because data is processed locally rather than transmitted externally, it also strengthens user privacy and data sovereignty (Tanwar et al., 2025), while reducing the ongoing bandwidth and server costs of cloud infrastructure.

2.7.3 Challenges

Edge devices, particularly microcontrollers, have far less memory, processing power, and battery capacity than cloud servers, limiting model size and complexity (Yahyati et al., 2025). Achieving a small enough footprint typically requires quantization, which can reduce accuracy compared to the original model (Alajlan & Ibrahim, 2022). Updating a model already embedded in a distributed app is also harder than updating a centralized cloud model, and local deployment introduces its own security concerns around protecting the model file from extraction or tampering.

2.7.4 Machine Learning Deployment on Mobile Edge Nodes

Deploying machine learning models locally on smartphones requires the trained model to be converted into a format the mobile device can execute efficiently. TensorFlow Lite serves this purpose. A model trained in TensorFlow is converted into a compressed .tflite file, where the model's numerical weights are reduced in precision through quantization. This makes the file small enough to be packaged inside a mobile app and fast enough to run without noticeable delay on a mid-range smartphone.

Much of the recent academic focus has been on running artificial intelligence directly on microcontrollers through a technique called TinyML. Bhattacharya et al. (2025) demonstrated this by deploying a health classification pipeline directly on an ESP32 microcontroller. While running inference on a microcontroller is an impressive engineering approach, it severely limits how complex the medical model can be, since microcontrollers have very restricted memory and processing capacity.

This study takes a more capable and scalable approach. Instead of forcing the machine learning model onto the constrained ESP32 hardware, the ESP32 acts strictly as a dedicated sensing and transmission unit. The actual anomaly detection runs within the Flutter mobile application on the user's smartphone. Modern smartphones possess substantially greater computational resources than microcontrollers. This allows the system to run more sophisticated anomaly detection models locally while still preserving strict data privacy and offline performance. Integrating local machine learning inference into a cross-platform Flutter application using TensorFlow Lite is a practical approach that remains underexplored in current academic literature.

2.8 Privacy, Security, and Data Sovereignty

The continuous monitoring of physiological data raises serious privacy concerns. Heart rates, movement patterns, and body temperatures can reveal sensitive details about a person's medical conditions and daily routines. When commercial wearables constantly transmit this raw data to third-party cloud servers, it creates opportunities for data breaches and unauthorized commercial exploitation.

Research by Cruz Castaneda and Bertemes Filho (2024) confirms that edge-based architectures substantially reduce these security risks. By processing raw data locally and keeping it on the user's device, the attack surface available to malicious actors is drastically minimized. This architectural choice also aligns with strict international data protection laws, such as the European Union General Data Protection Regulation and the Nigeria Data Protection Regulation. Both frameworks emphasize data minimization and local storage. By ensuring that the mobile application performs all critical inferences locally, the system guarantees that sensitive physiological metrics remain entirely under the patient's control.

2.9 Multi-Modal Sensing and Sensor Fusion

Relying on a single type of sensor for health alerts frequently leads to false positives. As Bent et al. (2020) demonstrated, PPG optical sensors are highly sensitive to movement and ambient light. A sudden arm movement can distort the optical signal, making it look like a dangerous cardiac event.

To solve this, modern systems must use multi-modal sensor fusion. By pairing the PPG sensor with a three-axis accelerometer, the system gains necessary context. If the accelerometer detects vigorous movement at the exact moment the heart rate reading becomes erratic, the system can mathematically filter out the motion artifact or safely ignore the reading. Adding a temperature sensor provides a third layer of context, helping to distinguish between normal physical exertion and a febrile response. Combining these three distinct data streams creates a much more reliable foundation for the mobile application's anomaly detection model to analyze.

2.10 Literature Mapping: Recent Advancements in Mobile Edge AI Health Monitoring

To systematically situate the research within the trajectory of recent scholarly development, Table 2.1 presents a structured mapping of ten representative studies published between 2020 and 2025.

Table 2.1: *Literature Mapping of Recent Advancements in Edge AI Health Monitoring*

SN	Authors	Addressed Challenge	Technique	Methodology	Identified Limitation	Solution
1	Bhattacharya et al. (2025)	High latency in cloud based health IoT.	TinyML Inference	Pipeline on ESP32.	Bandwidth bottlenecks and security risks.	A TinyML based wearable achieving real time anomaly detection.
2	Ghorbani et al. (2024)	Inter-subject variability in PPG signals.	Representation Learning	Transformation of raw PPG into compact features.	General AI models fail to account for individual baselines.	A two-stage framework for personalized

						anomaly detection.
3	Castañeda-Filho et al. (2024)	Privacy and security risks in chronic care.	Edge-IoT Driven AI	Three-layer architecture for health prediction.	Vulnerability of health data during cloud transit.	A decentralized privacy preserving framework for disease management.
4	Malche et al. (2024)	Hardware memory constraints on wearables.	TinyML Quantization	Deploying quantized neural networks on microcontrollers.	Model size exceeds the limits of typical hardware.	A low power system for tracking with a small memory footprint.
5	Pereira et al. (2024)	Lack of standardized Edge AI frameworks.	Systematic Mapping	Meta-analysis of 171 studies on edge integration.	Fragmented architectures for deploying AI.	A taxonomy and roadmap for future research in edge nodes.

6	Waldhauser et al. (2025)	False alarm rates in raw sensor data.	Wavelet Transform & ML	Architecture combining preprocessing with ML models.	Signal artifacts and noise in raw data streams.	A lightweight framework for high accuracy anomaly detection.
7	Bent et al. (2020)	Inaccuracy of optical heart rate sensors.	Validation Study	Systematic assessment of wearable accuracy.	Error rates increase during physical activity.	A call for context aware interpretation to correct PPG errors.
8	Yahyati et al. (2024)	Disconnect between cloud AI and low power IoT.	Systematic Review	Analysis of TinyML applications in healthcare.	High energy consumption of continuous cloud processing.	A review of optimization techniques for wearables.
9	Radhakrishnan et al. (2025)	Single sensor unreliability in health alerts.	Sensor Fusion	Integrating accelerometer and thermal data.	High rate of false positives using a single sensor.	A multi-modal fusion architecture for

						increased reliability.
10	Chen et al. (2025)	Complexity of real time PPG analysis.	ResNet- BiLSTM	A deep learning model for extracting features from PPG.	High computatio nal cost of deep modeling.	A lightweigh t hybrid model designed for real time PPG analysis.

2.11 Research Gap

A critical review of the literature reveals that while substantial progress has been made in wearable sensors, mobile edge computing, and signal processing individually, few studies have demonstrated a complete, end-to-end integration of these technologies into a unified, offline-first architecture. This study identifies three specific research gaps.

The first gap concerns the requirement for offline-first intelligence. Existing literature on medical decision support consistently treats internet connectivity as a mandatory requirement for generating health insights. Very few systems implement an architecture where core anomaly detection executes locally on the smartphone to guarantee offline safety, while treating cloud connectivity strictly as an optional enhancement for deeper conversational insights.

The second gap relates to the engineering of the mobile edge node. While deep learning models like ResNet and BiLSTM have demonstrated excellent accuracy for analyzing PPG data in controlled studies, there is a significant lack of research regarding how to practically deploy these models into cross-platform mobile environments using TensorFlow Lite. Existing studies either focus entirely on embedding tiny models onto limited microcontrollers or they focus solely on the mathematics of the algorithm. This project fills the gap by demonstrating a

complete integration pathway using Flutter and on-device inference to turn a standard smartphone into a capable medical edge node.

The third gap is the tension between data privacy and algorithmic personalization. Current privacy-preserving architectures secure data by processing it locally, but they often sacrifice analytical power by relying on basic, generalized thresholds that trigger false alarms. The system addresses this by showing that a mobile edge architecture can maintain absolute data privacy while simultaneously running a personalized anomaly detection model trained on WESAD wrist sensor data and further adapted to the individual user's own physiological history.

CHAPTER THREE

SYSTEM ANALYSIS AND DESIGN

3.0 Introduction

Research in software engineering often produces knowledge in one of two ways. The first way is by observing and explaining what already exists, which is the approach of natural or social science. The second way is by creating something new and evaluating whether it works, which is the approach of design science. This project belongs firmly in the second category. The goal is not simply to describe or analyze existing wearable systems but to design, build, and rigorously evaluate a new artifact that solves problems those systems have not addressed.

3.1 Methodology

For this reason, Design Science Research (DSR), as formalized by Peffers et al. (2007) and grounded in the three-cycle framework of Hevner (2007), has been selected as the guiding research methodology. DSR is a systematic, artifact-oriented approach that generates knowledge through the creation and evaluation of innovative solutions to real-world problems. It is particularly well-suited to engineering research because it explicitly requires that the artifact, in this case the wearable health monitoring system, be tied to a well-defined problem, built on prior knowledge from the literature, and evaluated against measurable criteria (Hevner et al., 2004).

The artifact produced by this research is an instantiation in DSR terminology, meaning a working prototype system. The dual outputs of the research are the artifact itself and the design knowledge gained from building and testing it, specifically the knowledge of how to integrate mobile edge AI, a personalized baseline engine, and a dual-mode conversational AI agent into a cohesive, offline-first health monitoring architecture.

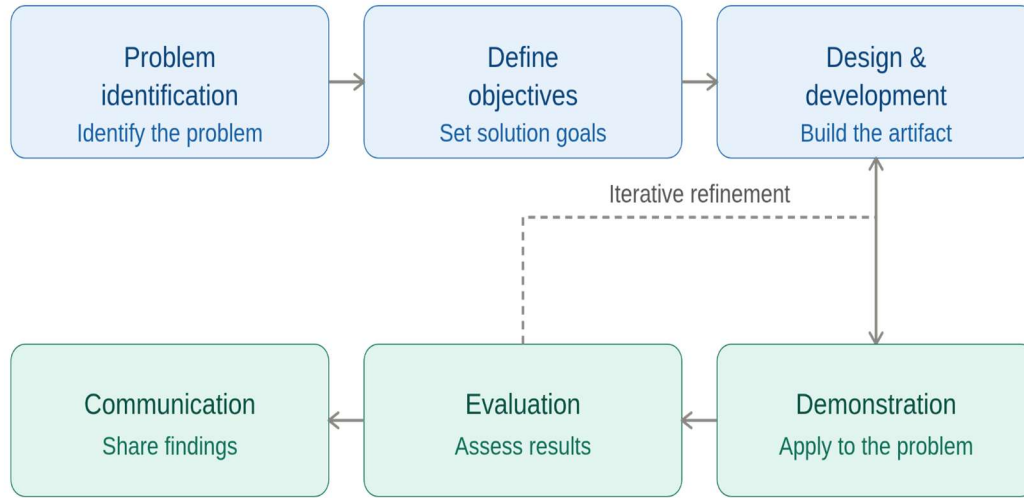


Figure 3.1: *DSRM diagram*

The diagram in Figure 3.1 illustrates the six sequential stages of the DSRM process as applied to this study: problem identification, defining solution objectives, design and development, demonstration, evaluation, and communication, with the dashed line showing the iterative loop back into design and development that occurred as the artifact was refined. This study entered the process at the problem-centered stage, since the research began from the cloud-dependency and generalized-model limitations established in Chapter One rather than from an existing solution or client request.

3.2 Research Design and Data Strategy

3.2.1 Nature of the Research

This research is applied and quantitative in character. It is applied because it targets a specific, practical problem rather than exploring a purely theoretical question. It is primarily quantitative because the evaluation criteria, inference latency, anomaly detection accuracy, F1 score, and BLE communication reliability, are all numerical measurements collected through controlled live testing. The research does not adopt a qualitative or survey-based approach because the

key questions about the artifact's performance are best answered through direct measurement rather than human opinion.

3.2.2 The Research Artifact

In DSR, the artifact is the central unit of the research. For this project, the artifact is a three-layer prototype system consisting of the ESP32 wrist-worn sensing device, the Flutter mobile application that serves as the mobile edge intelligence node, and the WESAD-trained TFLite anomaly detection model embedded within the application. The artifact also includes the personalized baseline engine and the dual-mode conversational AI agent, which together represent the system's most distinctive contributions.

3.2.3 Training Data Strategy

The machine learning component of the artifact required labeled physiological data for training. As established in Chapter Two, the WESAD dataset was selected for this purpose because its wrist device streams, including blood volume pulse at 64 Hz, body temperature at 4 Hz, and three-axis accelerometer data at 32 Hz, corresponded directly to the sensors on the ESP32 prototype. Only wrist device data was used. The classification task was binary: samples from the baseline condition formed the normal class, while samples from the stress condition formed the anomaly class.

3.2.4 Evaluation Strategy

The artifact was evaluated through live hardware testing rather than simulation, which aligned with DSR's requirement that the demonstration and evaluation phases involve the artifact in a realistic context. A test subject wore the assembled ESP32 prototype during different activity states, including rest, mild physical movement, and stress-inducing mental tasks. The deployed TFLite model's outputs were observed in real time on the Flutter application. The evaluation measured anomaly detection accuracy and F1 score on the live test data, inference latency from data receipt to detection result, offline AI agent response time, and BLE communication reliability.

3.3 Machine Learning Pipeline

The machine learning component of this system follows a structured pipeline that begins with dataset acquisition and ends with a deployed model inside the Flutter application.

3.3.1 Dataset Acquisition

The WESAD dataset was obtained from the UCI Machine Learning Repository, where it is publicly available under an open access license. The dataset was originally published by Schmidt et al. (2018) and contains physiological and motion recordings from 15 participants wearing both a chest device and a wrist device during a controlled laboratory study. Only the wrist device data from the Empatica E4 was used in this project. The wrist streams relevant to this study were blood volume pulse sampled at 64 Hz, body temperature sampled at 4 Hz, and three-axis accelerometer data sampled at 32 Hz. These corresponded directly to the MAX30102, DS18B20, and MPU6050 sensors on the ESP32 prototype.

The classification task was structured as a binary problem. Samples labelled as the baseline condition formed the normal class, representing a participant's resting physiological state. Samples labelled as the stress condition formed the anomaly class, representing a state of acute physiological deviation induced through the Trier Social Stress Test. The amusement condition was excluded from training to keep the model focused on the clinically relevant distinction between normal and anomalous physiological states.

3.3.2 Data Cleaning

Before any features were extracted, the raw wrist signals underwent a cleaning stage to remove noise and ensure consistency across all 15 subjects. The BVP signal was filtered using a Butterworth bandpass filter with cutoff frequencies selected to retain the physiologically meaningful frequency range of the cardiac pulse, typically between 0.5 Hz and 4 Hz, while rejecting high-frequency electrical noise. The accelerometer data had a magnitude signal computed from the three raw axes using the Euclidean norm, providing a single movement intensity value that simplified the motion artifact correction step later in the pipeline. Temperature readings, which are already low-frequency signals, were checked for sensor dropout or physically implausible values, and any affected samples were excluded. After per-

signal cleaning, the data from all 15 subjects were synchronized, and the labels from the original dataset were mapped to the binary normal and anomaly classes defined above.

3.3.3 Feature Extraction

The cleaned time-series data were segmented into fixed-length windows using a sliding window approach. Each window represented a short observation period of the user's physiology, and the model learned to classify whether the signals within that window represented a normal or anomalous state. Window boundaries overlapped to ensure that transitions between states were captured adequately without losing information at segment boundaries.

From each BVP window, heart rate variability features were derived by detecting the peaks of the pulse waveform and computing the intervals between successive peaks. From each temperature window, statistical descriptors such as the mean and rate of change were extracted. From each accelerometer window, the mean and standard deviation of the magnitude signal were computed to quantify movement intensity. These features were assembled into a structured input vector for each time window across all subjects, producing the training dataset that was fed into the model.

3.3.4 Model Training

The anomaly detection model was trained as a binary classifier on the feature vectors extracted from the WESAD wrist data. A Long Short-Term Memory network was used as the primary architecture, selected for its ability to capture temporal dependencies in time-series physiological signals without the computational demands of a full ResNet-BiLSTM architecture that would be difficult to compress for mobile deployment. The model was trained using leave-one-subject-out cross-validation, where data from 14 subjects were used for training and the remaining subjects data were used for testing, rotating through all 15 subjects. This approach provided an estimate of how well the model generalized to individuals whose data it had not seen before, which is the relevant scenario for a new user setting up the device for the first time.

The training objective was to minimize binary cross-entropy loss. Since the baseline condition occupied a larger proportion of the WESAD recording time than the stress condition, class weighting was applied during training to prevent the model from learning to simply predict the normal class for every input. Model performance was evaluated using accuracy, precision, recall, and the F1 score on the held-out test subject in each cross-validation fold. The F1 score was given particular importance because it balanced the cost of missing a genuine anomaly against the cost of generating a false alarm.

3.3.5 Model Validation

Beyond cross-validation on the WESAD dataset, the trained model was validated through live hardware testing after the prototype was assembled. The ESP32 wearable was worn by a test subject while different activities, including rest, mild physical exertion, and stress-inducing tasks, were performed, and the deployed model's outputs were observed to assess whether the anomaly detection logic behaved as expected in a real use environment. This live validation step also measured inference latency on the target Android device to confirm that the system met the 200 millisecond response time requirement defined in the non-functional requirements.

3.3.6 Model Deployment Using TensorFlow Lite

Once training was complete and the model had been validated, the final model was exported from TensorFlow as a SavedModel and then converted into the TensorFlow Lite format using the TFLiteConverter. During conversion, post-training quantization was applied to reduce the model's weight precision from 32-bit floating-point numbers to 8-bit integers. This reduced the file size significantly and sped up inference on mobile hardware without a meaningful loss in detection accuracy. The resulting .tflite file was placed in the Flutter application's assets directory and loaded at runtime by the tflite_flutter package, which provides the Dart interface for running TFLite models inside a Flutter app. At inference time, each incoming normalized sensor window was passed to the model interpreter, and the output score was compared against the anomaly threshold to determine whether an alert should be triggered.

3.4 Analysis of the Existing System

3.4.1 Description of Existing Systems

Contemporary wearable health monitoring systems, represented by commercial products from Apple, Samsung, Garmin, and Fitbit, follow a consistent cloud-dependent pipeline. The wearable device collects raw sensor data and buffers it locally. A companion smartphone application receives this data over Bluetooth, displays generic charts and metrics, or forwards it to a manufacturer-operated cloud server where machine learning models perform the actual analysis using aggregated population data. The results are then returned to the smartphone for display.

3.4.2 Shortcomings of the Existing System

A thorough analysis reveals five structural limitations that significantly undermine the reliability and usefulness of existing wearable systems, particularly in contexts like Nigeria where both infrastructure and individual health needs vary greatly.

The first and arguably most fundamental limitation is the use of generalized, population-based machine learning models. Commercial wearables train their detection algorithms on aggregated data from millions of users to establish what is statistically normal across a broad population. However, physiological signals are highly variable between individuals. Ghorbani et al. (2024) demonstrated that these generalized models achieve anomaly detection accuracy barely above random guessing when evaluated at the individual level, while models trained on a user's own historical data achieved near-perfect accuracy on the same test cases. This means current wearables frequently generate false alarms for healthy users while simultaneously missing real anomalies in individuals whose normal physiology falls outside the population average.

The second limitation is the complete absence of a conversational AI agent. Existing wearable systems are entirely passive: they display charts and occasionally push a notification, but they offer no mechanism for the user to ask questions, seek guidance, or receive context-aware explanations of their health data. A user who receives a high heart rate alert has no way to ask the system whether the reading is serious, what factors might have caused it, or what they should do next.

The third limitation is the dependency on internet connectivity for advanced analysis. As established in Chapter One, cloud-dependent systems lose their analytical capabilities entirely when network access is unavailable. The fourth limitation is the time delay introduced by sending data to a remote server and waiting for a response, which can range from hundreds of milliseconds to several seconds depending on network conditions. For time-sensitive medical events, this latency is unacceptable. The fifth limitation is the privacy risk of continuously transmitting raw physiological data to third-party servers, which creates exposure to data breaches and unauthorized commercial exploitation (Cruz Castaneda and Bertemes Filho, 2024). Together, these five limitations define the design problem that the artifact is built to solve.

3.5 System Design

3.5.1 System Overview

The system relocated all analytical intelligence from the cloud to the user's smartphone, treating the wearable hardware unit strictly as a data acquisition and transmission device. The Flutter mobile application served as the mobile edge intelligence node, receiving raw sensor data over BLE, processing it locally, running a personalized anomaly detection model built from the individual user's own physiological history, and generating health alerts without any network dependency. A key feature that distinguishes this system from existing wearables is its embedded dual-modal AI agent. In offline mode, this agent is a rule-based decision support module that interprets anomaly detection results and responds to the user's natural language queries through a conversational interface. In online mode, the agent connects to an external large language model API, unlocking richer and more nuanced conversational responses. The cloud never receives raw sensor readings under any circumstances. Only anonymized summary statistics are shared with the online AI service when the user actively initiates a query.

3.5.2 Functional Requirements

The functional requirements below are derived from the project objectives stated in Chapter One and the research gaps identified in Chapter Two. They define every action the system must be capable of performing.

Table 3.1: *Functional Requirements of the System*

S/N	Requirement Description	Components
1	Capture PPG, temperature, and accelerometer data at configurable sampling intervals	ESP32 Hardware
2	Transmit sensor data via BLE to the mobile application without requiring internet access	ESP32 + Flutter BLE Module
3	Display real-time physiological metrics on an interactive dashboard	Flutter UI Layer
4	Store historical sensor readings in a local on-device SQLite database	Flutter Data Layer
5	Build and update a personalized physiological baseline from the user's own stored readings	Flutter Baseline Engine
6	Execute on-device anomaly detection using the personalized baseline and a compressed TFLite model	Flutter AI Inference Layer
7	Generate immediate visual and audible alerts when an anomaly is detected, fully offline	Flutter Alert Engine
8	Provide an offline AI agent that interprets alerts and responds to health queries using rule-based logic	Flutter Offline Decision Support Module
9	Provide an enhanced AI agent powered by an external LLM API when internet access is available	Flutter Cloud AI Module
10	Allow the user to review a timestamped log of all past readings and alerts	Flutter History View

3.5.3 Non-Functional Requirements

Non-functional requirements define the quality, performance, and operational constraints the system must satisfy. Offline availability is a non-negotiable constraint: every core monitoring, detection, alerting, and AI agent function must operate without any internet connection. Dashboard updates must appear within 500 milliseconds of data receipt, and anomaly detection inference must complete within 200 milliseconds on a mid-range Android device. The AI agent's offline response must generate within one second of the user's query submission. No raw physiological data may be transmitted to any external server at any time. The application was deployed on Android 8.0 and above, and the hardware battery must sustain at least eight hours of continuous operation on a single charge.

Table 3.2: *Non-Functional Requirements of the System*

S/N	Requirement	Metric / Constraint
1	Offline availability	100% core functionality without internet, including AI agent
2	Dashboard latency	Live readings displayed within 500ms of data receipt
3	Inference speed	Anomaly detection result generated within 200ms on mid-range Android
4	Offline AI response time	Rule-based agent response within 1 second of query submission
5	BLE reconnection	Automatic reconnection within 10 seconds of link drop
6	Data privacy	Zero raw sensor data transmitted to any external server at any time
7	Android compatibility	Android 8.0 and above
8	Battery life	Minimum 8 hours continuous hardware operation per charge

9	Local data retention	Minimum 30 days of readings stored locally without overflow
---	----------------------	---

3.5.4 Hardware Component Design

The wearable sensing unit was built around the ESP32 development board, selected for its dual-core 240 MHz processor, 520 kilobytes of static RAM, and native Bluetooth Low Energy 4.2 support integrated on a single chip (Espressif Systems, 2023). The photoplethysmography sensor was the MAX30102, which integrates red and infrared LEDs alongside a photodetector in a single compact module. It communicated with the ESP32 over the I2C bus, and the firmware sampled its output at 100 Hz before applying a digital low-pass filter to attenuate high-frequency noise. Motion data were captured by the MPU6050 six-axis inertial measurement unit, connected on the same I2C bus. Its three-axis accelerometer output served two functions: providing direct information about the user's physical activity level as a physiological signal, and supplying the motion context the anomaly detection model needed to correctly interpret PPG readings during physical exertion. Body surface temperature was measured by the DS18B20 digital temperature sensor, which communicates over the Dallas 1-Wire protocol. The complete circuit was powered by a 3.7 V lithium-polymer battery, with a TP4056-based module handling battery charging via USB and a dedicated 3.3 V low-dropout voltage regulator providing a stable supply voltage to the ESP32 and all sensor modules.

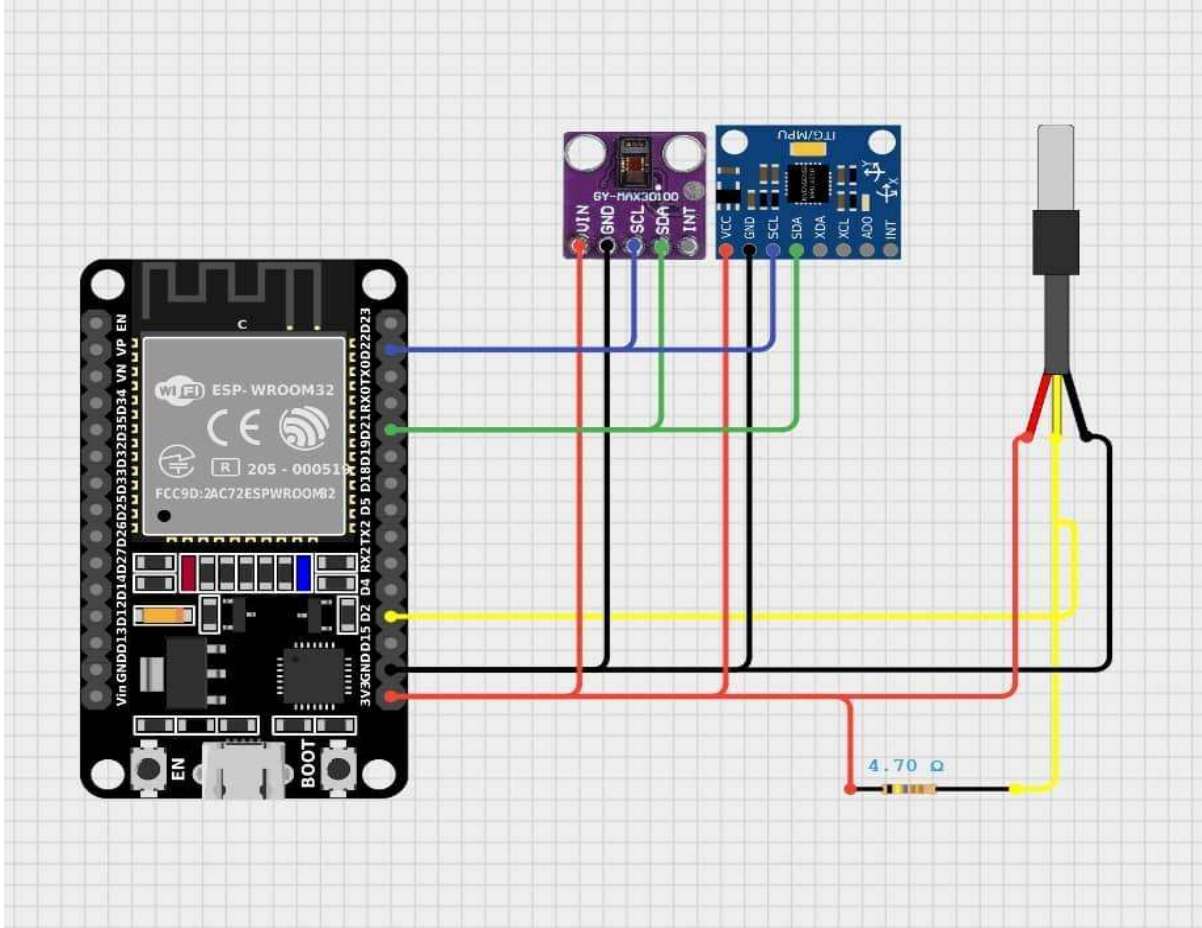


Figure 3.2: *Circuit diagram*

This Circuit in Figure 3.2 visualizes the sensor wiring described above in schematic form, showing how the MAX30102, MPU6050, and DS18B20 connect to the ESP32's I2C bus and digital pins

3.5.5 Mobile Application Design

The mobile application was built using Flutter, which compiles Dart source code directly into native machine code for Android. Flutter was selected over alternatives such as React Native because its native compilation model delivered the real-time processing performance required for concurrent BLE data reception, local inference, and AI agent query handling. The application was structured into four logical layers. The presentation layer contained all visible interface elements, including the live sensor charts, the anomaly alert overlay, the AI agent conversation screen, and the historical readings view. The business logic layer coordinated all

application state using the BLoC pattern, managing the BLE connection lifecycle, computing derived metrics, triggering inference cycles, and routing user queries to either the offline decision support module or the online LLM client depending on network availability. The data layer handled all local storage through SQLite via the sqflite package. The hardware interface layer manages all BLE communication through the flutter_blue_plus package.

3.6 System Architecture and Models

3.6.1 Three-Layer System Architecture

The system was organized into three layers, each with a clearly defined responsibility and interface boundary. The Hardware Sensing Layer at the base encompassed the ESP32 and all attached sensors. This layer carried no analytical intelligence; its sole function was accurate physiological data acquisition and reliable BLE packet transmission. Keeping this layer minimal made the wearable unit power-efficient, straightforward to manufacture, and easy to replace or upgrade without affecting any higher layer. The Mobile Edge Layer ran entirely on the user's smartphone as the Flutter application. This layer received raw data from the hardware layer, processed and stored it locally, maintained the personalized physiological baseline, executed the anomaly detection model, and drove the AI agent interface. All of the system's intelligence resided here, which made the system capable of full offline operation.

3.6.2 Use Case Model

The system had one primary actor, the User, and two secondary actors: the ESP32 Wearable Device, which acted as an automated data source, and the Cloud AI Service, which served as an optional external provider. The User's main interactions with the system included scanning for and pairing with the wearable device; starting and stopping a monitoring session; viewing the real-time health dashboard; receiving and acknowledging anomaly alerts; querying the offline AI agent for guidance and explanations; reviewing the full history of past readings and alerts; and querying the online AI agent for richer analysis when a network connection was present. The Cloud AI Service was associated only with the online AI query use case, which carried an include relationship to an internet connectivity check, making clear that this path was unavailable when the system was in offline mode.

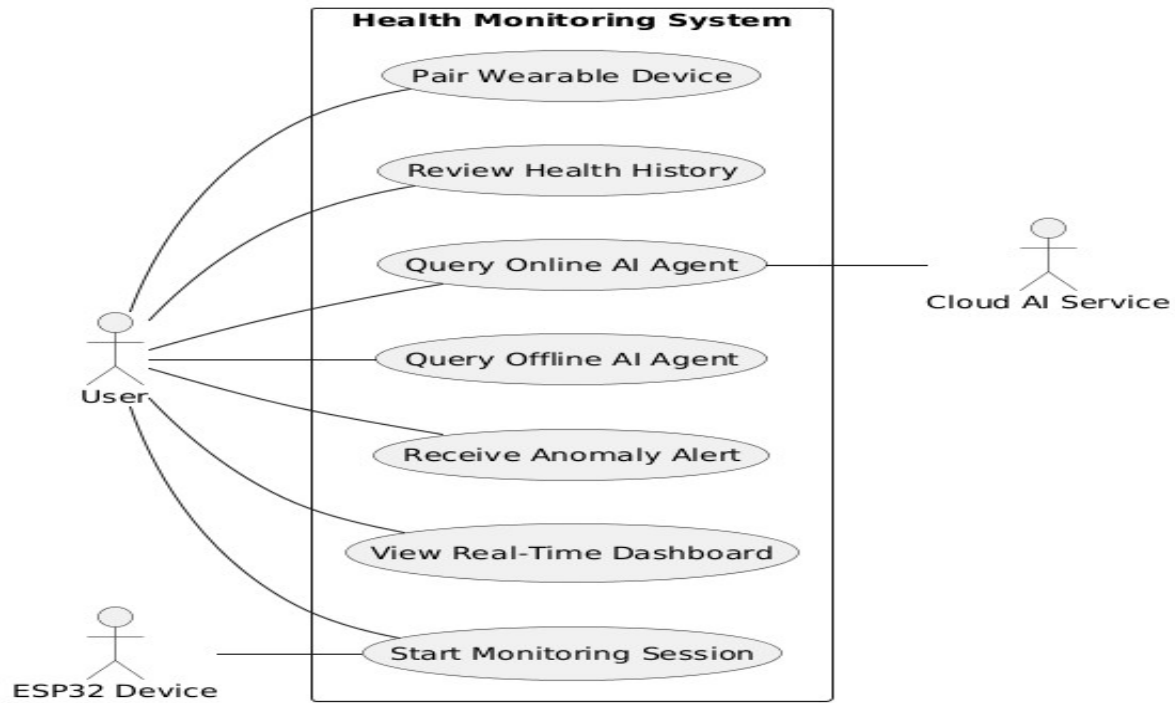


Figure 3.3: UML Use Case Diagram Showing System Actors and Use Cases

3.6.3 Class Design

The mobile application is organized around nine primary classes. The `SensorDevice` class holds the BLE device identifier and connection state. The `BLEManager` class handles all Bluetooth operations and produces a stream of raw byte arrays passed to the `PacketParser`, which decodes them into typed `SensorReading` objects carrying a timestamp alongside PPG amplitude, temperature, and three-axis accelerometer values. The `HealthDataStore` class persists readings to SQLite and maintains the in-memory baseline window that the `BaselineEngine` class uses to continuously update the user's personal physiological normal ranges. The `AnomalyDetector` class normalizes each incoming reading against the current baseline before invoking the TFLite model interpreter; when the resulting score exceeds a configured threshold, it notifies the `AlertEngine`, which determines alert priority and triggers the appropriate user notification. The `DecisionSupportAgent` class handles both offline and online AI interactions: in offline mode it applies a rule tree against the alert context and the user's recent readings to generate structured guidance; in online mode it serializes an

anonymized health summary and dispatches it to the CloudClient class, which manages HTTPS communication with the LLM API. The DashboardController class coordinates the entire presentation layer by subscribing to data events from BLEManager, alerts from AlertEngine, and agent responses from DecisionSupportAgent.

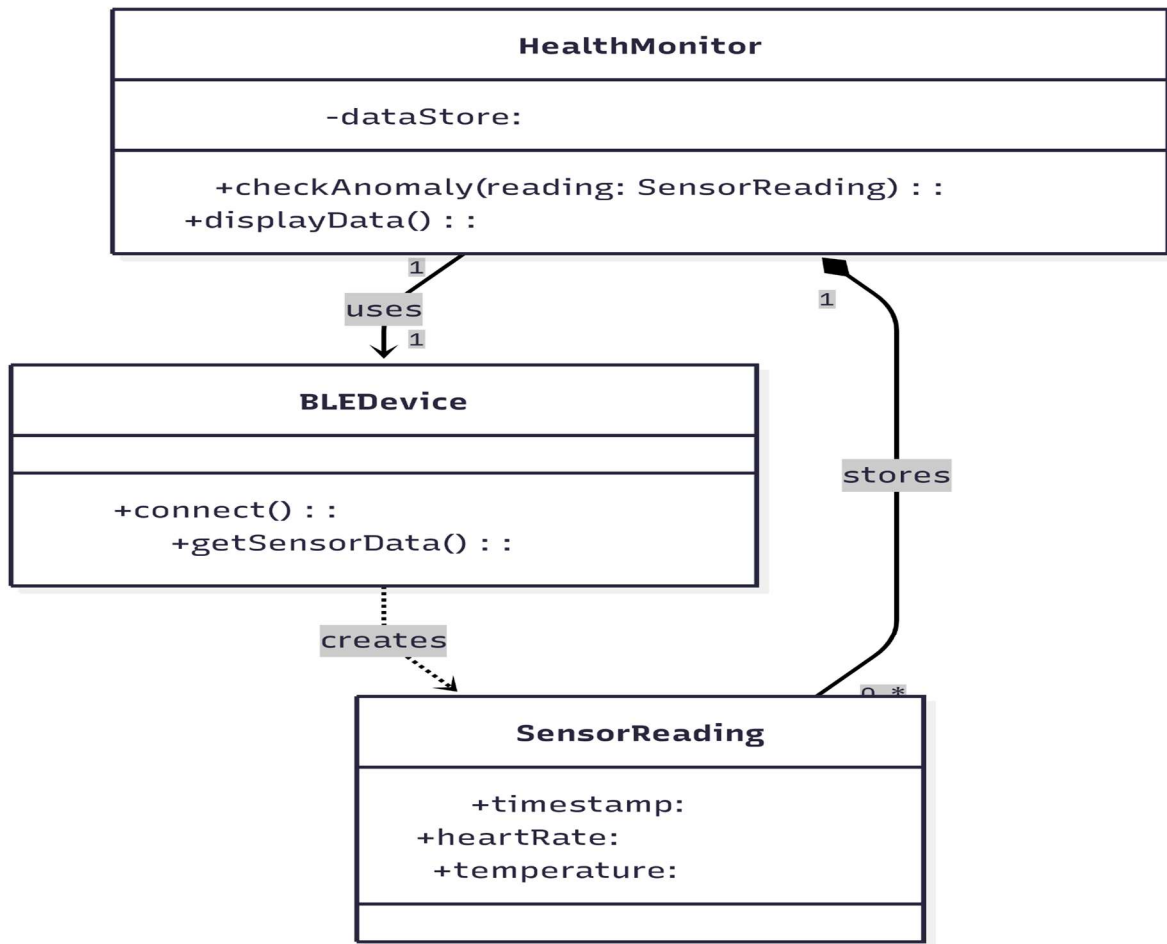


Figure 3.4: UML Class Diagram of the Flutter Mobile Application

3.7 Design Flowcharts and Algorithms

3.7.1 Main System Operation Flowchart

When the application launches, it checks local preferences for a previously paired device address. If one is found, it attempts automatic reconnection; otherwise it presents the BLE scanning interface. Once a connection is established, the application subscribes to the

notification characteristic and enters the monitoring loop, which decodes each incoming packet, persists it to the database, updates the baseline window, refreshes the dashboard, and evaluates the reading for anomalies. A detected anomaly generates a timestamped alert, which is recorded in the local database and displayed to the user through the alert overlay. If the user opens the AI agent from the alert overlay, the conversation interface appears and the DecisionSupportAgent provides guidance. If the BLE connection drops at any point, the application enters an automatic reconnection state without any interruption to the data stored so far. The loop continues until the user explicitly stops the session.

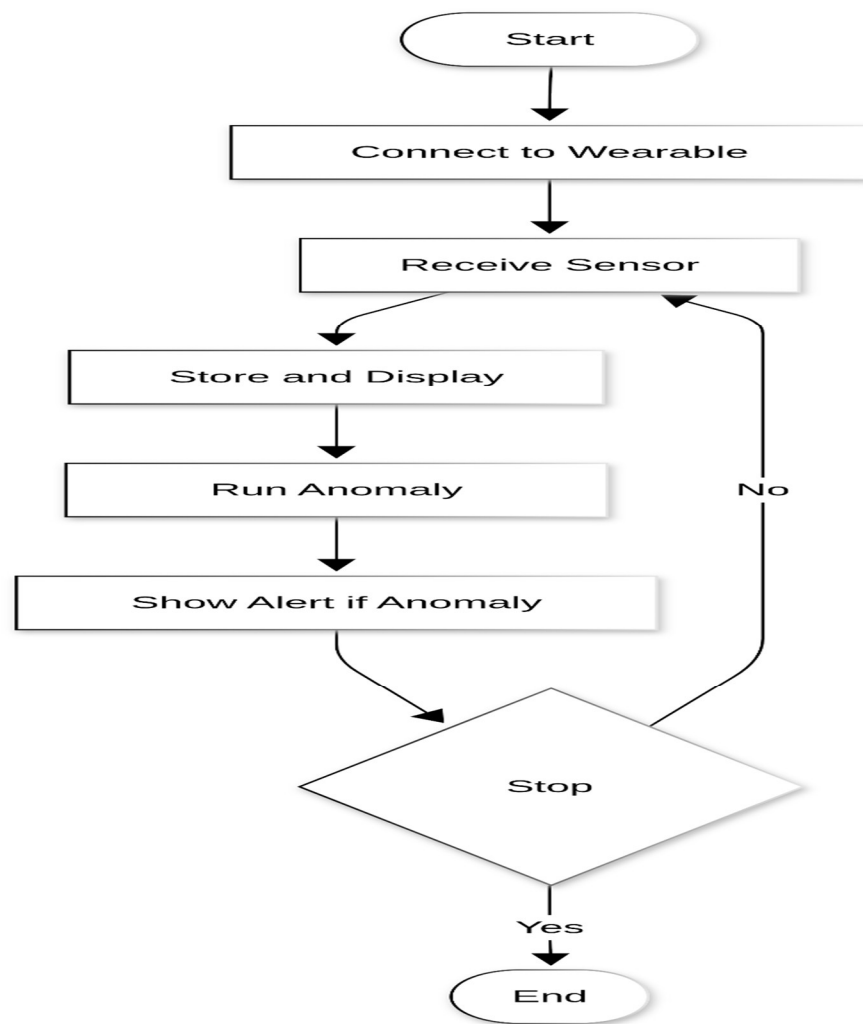


Figure 3.5: *Main System Operation Flowchart*

3.7.2 BLE Data Transmission Protocol

The ESP32 firmware advertises a custom GATT service identified by a 128-bit UUID unique to this application. A single notify characteristic carries all sensor data as a fixed 16-byte binary packet. The first four bytes carry a millisecond-precision timestamp, the next four carry the raw PPG amplitude as a 32-bit integer, the following two carry the temperature reading scaled to two decimal places of precision, and the remaining six bytes carry the three accelerometer axis values. The Flutter PacketParser extracts each field from the incoming byte array using Dart's ByteData API at the exact offsets defined in this protocol.

3.8 Advantages of the System

The artifact offers meaningful improvements over existing cloud-dependent systems across every dimension identified as deficient in the earlier analysis. Table 3.4 presents a structured comparison.

Table 3.3: *Comparative Analysis of Existing System Against the System*

Feature	Existing System	Developed System
Anomaly detection model type	Generalized population-based model	Personalized model built from the individual user's own historical data
AI conversational agent	Not available on any existing commercial wearable	Available in both offline (rule-based) and online (LLM-powered) modes
Internet dependency for core functions	Required at all times; system fails without network	Not required; full monitoring and AI agent operate offline
Alert latency	Seconds, due to cloud round-trip	Under 200 milliseconds, processed locally on the smartphone

Data privacy	Raw physiological data transmitted to third-party cloud servers	All raw data remains on the user's device at all times
Hardware cost	High; proprietary commercial wearable devices	Low; open-source ESP32 hardware and freely available sensors
Multi-sensor context awareness	Limited; PPG errors during motion often uncorrected	Accelerometer provides motion context to filter PPG artifacts in real time
Usability in low-connectivity regions	Severely limited or completely non-functional	Fully functional, including AI agent, without any network access
System intelligence location	Cloud server; inaccessible when offline	User's smartphone; always accessible regardless of connectivity

Beyond the advantages captured in Table 3.3, the two contributions that most clearly distinguish this system from current offerings are the personalized baseline engine and the offline conversational AI agent. Together, these features address the most significant limitations that studies such as Ghorbani et al. (2024) and Pereira et al. (2024) have consistently identified as unresolved: the failure of population-based models to detect individual-level anomalies, and the absence of any mechanism for users to receive context-aware health guidance without a network connection. The system also demonstrates that these capabilities can be achieved using low-cost, open-source hardware and freely available tools, which has direct implications for healthcare access in resource-limited environments.

CHAPTER FOUR

SYSTEM IMPLEMENTATION

4.0 Introduction

This chapter presents the implementation of the Edge AI Wearable Health Monitoring System described in Chapter Three, covering the development tools, hardware assembly, BiLSTM anomaly detection model, Flutter mobile application, system integration, and testing carried out against the stated requirements. Both the hardware prototype and the mobile application have been fully implemented and tested as a working artifact, consistent with the Design Science Research approach adopted for this project. The implementation follows the three-layer architecture from Chapter Three: the ESP32 wearable as the sensing module, the Flutter application as the mobile edge intelligence module, and the TensorFlow Lite model as an on-device inference asset.

4.1 System Implementation

4.1.1 Development Environment and Tools

A combination of hardware development tools, mobile development tools, and machine learning tools was used to build the system. Table 4.1 summarizes the tools and their roles in the project. The S/N column provides a serial number for each entry, the Tool column names the specific software or environment used, and the Role in the Project column describes the exact function that tool performed during the implementation of this system.

Table 4.1: *Development Tools and Their Roles*

S/N	Tool	Role in the Project
1	Arduino IDE	Writing, compiling, and uploading the ESP32 firmware, including sensor drivers and BLE communication code
2	Visual Studio Code	Primary code editor for the Flutter application's Dart source files across all layers
3	Flutter SDK and Dart	Building the cross-platform mobile edge intelligence application

4	Colab	Cloud environment for the WESAD data cleaning, feature extraction, model training, and TensorFlow Lite conversion pipeline
5	TensorFlow and TensorFlow Lite Converter	Building, training, and quantizing the BiLSTM model into a mobile-ready .tflite file
6	Android Studio	Managing the Android emulator and physical device testing
7	Git and GitHub	Version control for the firmware and Flutter application source code

4.1.2 Wearable Hardware

The wearable sensing unit was assembled according to the hardware design in Chapter Three, with the ESP32 development board as the central controller, the MAX30102 pulse oximeter and heart rate sensor, the MPU6050 six-axis accelerometer and gyroscope, and the DS18B20 digital temperature sensor as the three primary sensing components. Power was supplied through a 3.7V lithium polymer battery with a TP4056 charging module and a 3.3V voltage regulator. The MAX30102 and MPU6050 communicate over I2C on a shared bus, while the DS18B20 uses the Dallas 1-Wire protocol on a separate digital pin with a pull-up resistor.

Table 4.2: *ESP32 Pin Connections for the Wearable Sensing Unit*

S/N	Component	Pin on Component	Connected ESP32 Pin	Notes
1	MAX30102	VIN	3.3V	Sensor power supply
2	MAX30102	GND	GND	Common ground
3	MAX30102	SDA	GPIO 21	I2C data line
4	MAX30102	SCL	GPIO 22	I2C clock line
5	MPU6050	VCC	3.3V	Sensor power supply
6	MPU6050	GND	GND	Common ground
7	MPU6050	SDA	GPIO 21	I2C data line
8	MPU6050	SCL	GPIO 22	I2C clock line
9	DS18B20	VDD	3.3V	Sensor power supply

10	DS18B20	GND	GND	Common ground
11	DS18B20	DQ	GPIO 4	1-Wire data line, 4.7k pull-up to 3.3V
12	TP4056 Module	OUT+ / OUT-	VIN / GND	Power input to the ESP32 board

Table 4.2 presents the complete wiring configuration of the assembled prototype. The S/N column provides a serial number, the Component column identifies the sensor or module, the Pin on Component column names the specific pin on that sensor, the Connected ESP32 Pin column specifies which GPIO or power pin on the ESP32 board receives that connection, and the Notes column describes the electrical role of each connection.

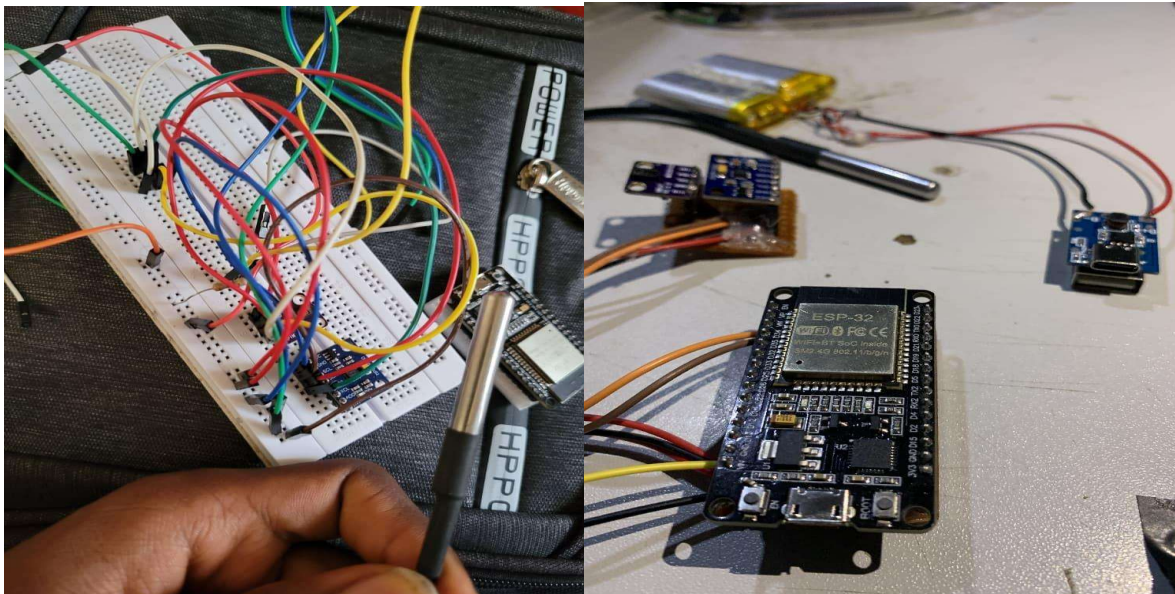


Figure 4.1: *Development.*

Figure 4.1 above illustrates the breadboard development stage of the wearable hardware. The image shows the ESP32 development board at the center, with the MAX30102 pulse oximeter sensor connected on the I2C bus at GPIO 21 and GPIO 22, the MPU6050 accelerometer and gyroscope sharing the same I2C bus, and the DS18B20 temperature sensor connected to GPIO 4 with its pull-up resistor visible. The jumper wires running between the breadboard rails and the sensor modules represent the connections defined in Table 4.2



Figure 4.2: *Fully assembled wrist-worn device*

4.1.3 Firmware and BLE Interface

The ESP32 firmware was written in C++ using the Arduino IDE and Arduino core for ESP32. It initializes the I2C bus for the MAX30102 and MPU6050, configures the 1-Wire interface for the DS18B20, and continuously samples all three sensors, packaging the readings into binary BLE packets containing a timestamp, physiological readings, and motion values. The firmware also configures the ESP32 as a BLE peripheral advertising a custom GATT service; once the Flutter application subscribes to the notification characteristics, it streams live sensor frames continuously without requiring an internet connection.

4.1.4 Flutter Mobile Application

The Flutter mobile application was implemented as the main mobile edge intelligence node of the system, using a routed shell with major screens for the live dashboard, history, alerts, chat assistant, and settings. GoRouter handles navigation, Riverpod coordinates application state and service providers, and reusable widgets provide a consistent interface.

The BLE communication module, implemented using the `flutter_blue_plus` package, scans for the PulseEdge device name or service UUID, connects to the wearable, and subscribes to notification characteristics for PPG, temperature, IMU, and device status frames. Incoming byte payloads are decoded into typed sensor frames and sent into the storage and feature

extraction pipeline; if the connection drops, the service enters a reconnection state and attempts to restore the link.

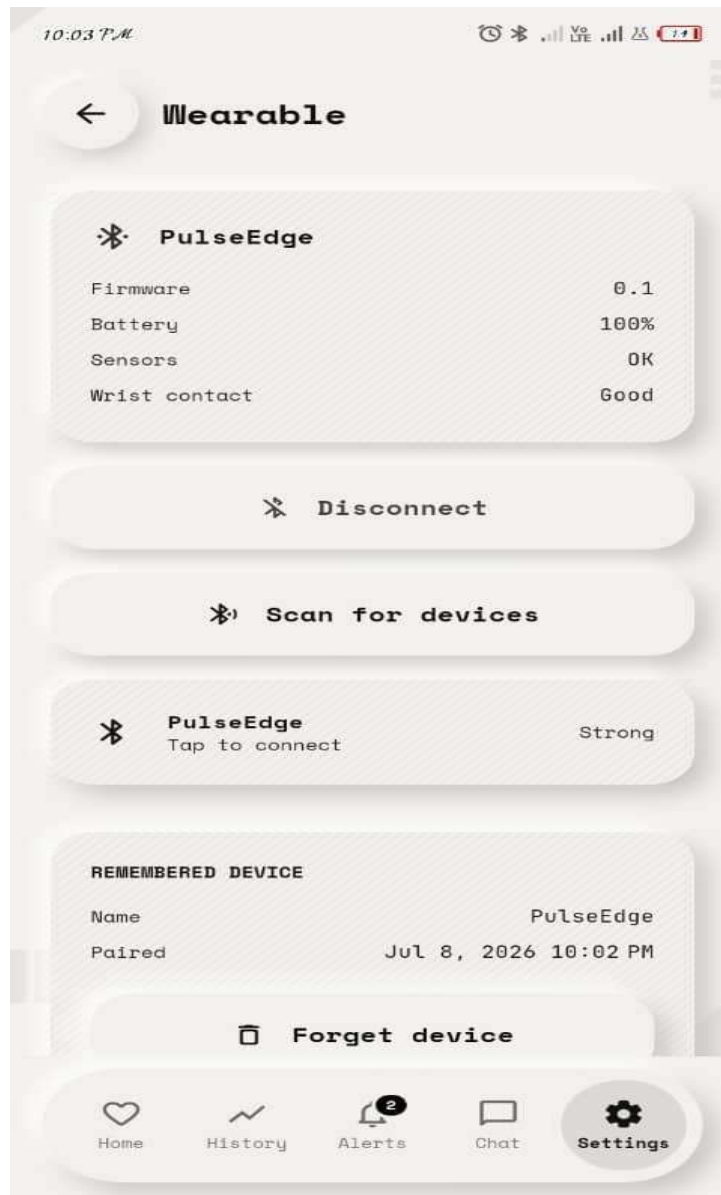


Figure 4.3: Mobile app screen showing BLE device scanning and pairing with the wearable device

Figure 4.3 above illustrates the BLE device scanning screen of the PulseEdge mobile application. The screen lists discovered BLE peripherals within range, identified by their device name and signal strength indicator. Tapping the PulseEdge device name in the list initiates the connection sequence, during which the application subscribes to the PPG,

temperature, IMU, and status notification characteristics exposed by the ESP32 firmware. The connection status badge at the top of the screen transitions from Scanning to Connected once the pairing is complete. The real-time dashboard displays heart rate, oxygen level, skin temperature, movement, rotation, activity state, battery status, and BLE connection state, updating from live streams exposed by the BLE service.

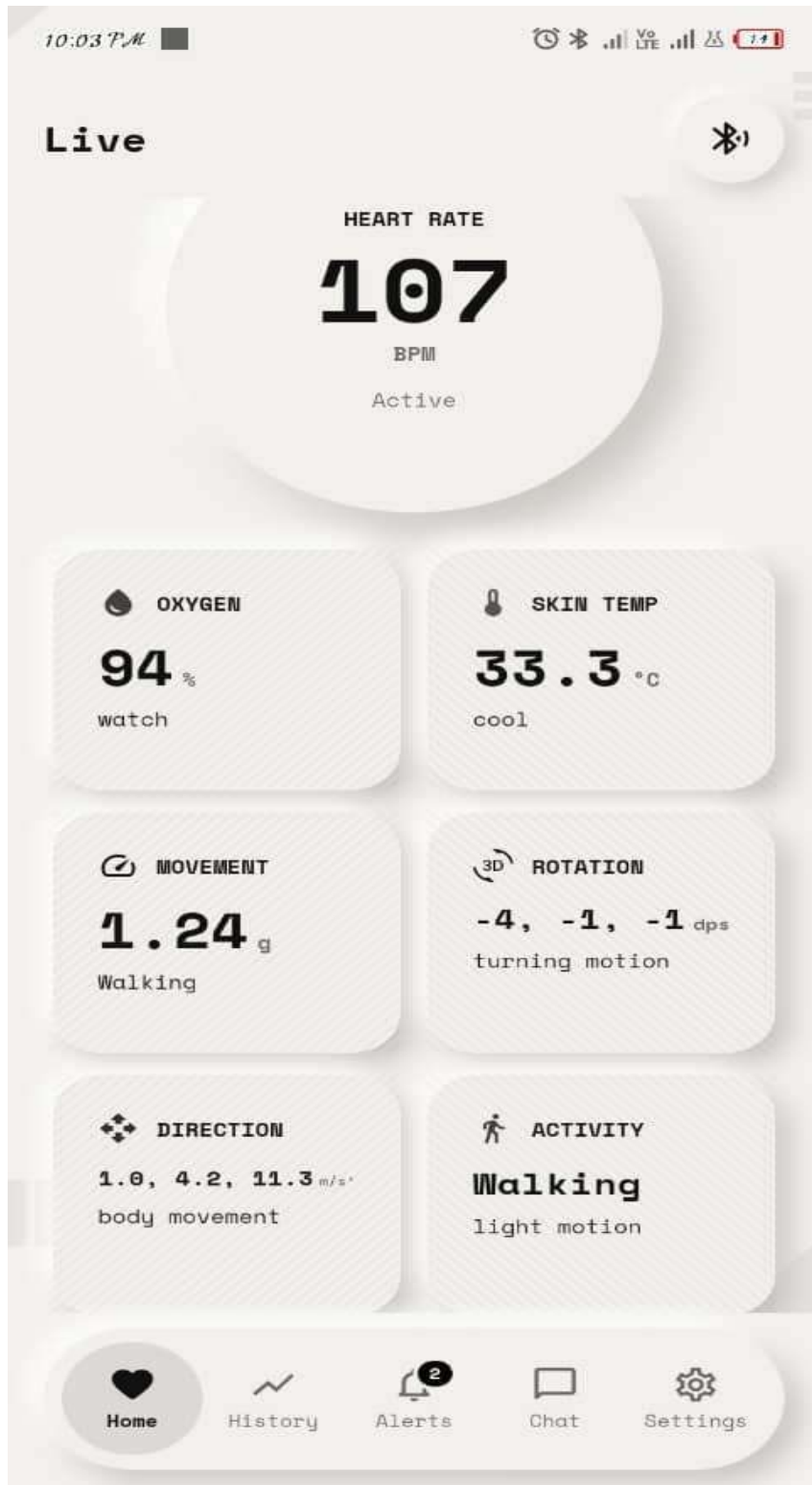


Figure 4.4: Mobile app dashboard showing live sensor readings

Figure 4.4 above illustrates the live monitoring dashboard of the Flutter mobile application. The top row of metric cards displays the current heart rate in beats per minute, the estimated blood oxygen saturation percentage, and the current skin surface temperature in degrees Celsius, each updating in real time as packets arrive from the wearable. The middle section shows the accelerometer-derived activity state, the battery level of the ESP32 device, and the BLE connection indicator. A scrollable chart below the cards renders the recent PPG signal waveform as a continuous line graph, allowing the user to observe the pulse shape in real time. All sensor readings are persisted locally using Drift over SQLite, storing paired device records, sensor samples, extracted feature rows, anomaly alerts, baseline statistics, chat messages, and profile information. The baseline service updates user-specific statistics from these feature windows, grouped by metric, time of day, and activity level.

The on-device anomaly detection flow is implemented inside the Flutter app: the feature extractor aggregates incoming sensor frames into feature windows, the baseline service updates the user's personal statistics, and the `AnomalyDetector` class loads the bundled model using `tflite_flutter`, returning an anomaly probability between 0 and 1 that is passed to the `AlertEngine` for alert decision-making.

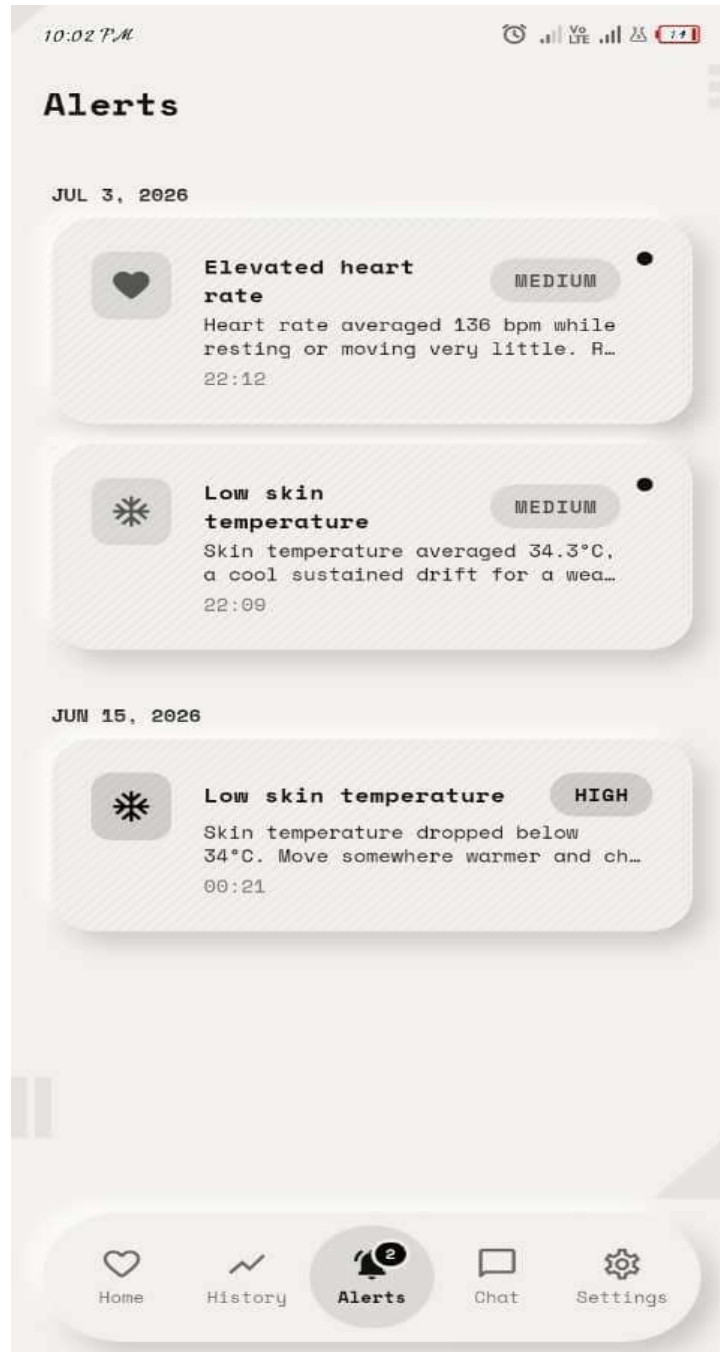


Figure 4.5: *Mobile application anomaly detection alert overlay*

Figure 4.5 above illustrates the anomaly alert overlay that appears on the dashboard when the AnomalyDetector flags a reading as anomalous. The overlay card displays the timestamp of the detected event, the specific metric that triggered the alert, the current value compared against the user's personal baseline, and the anomaly probability score returned by the TFLite

model. Two action buttons appear at the bottom of the overlay: one to dismiss the alert and one to open the AI agent for contextual guidance. All of this processing and display occurs entirely on-device without any internet connection. The decision support interface is implemented through the app's chat and assistant services.

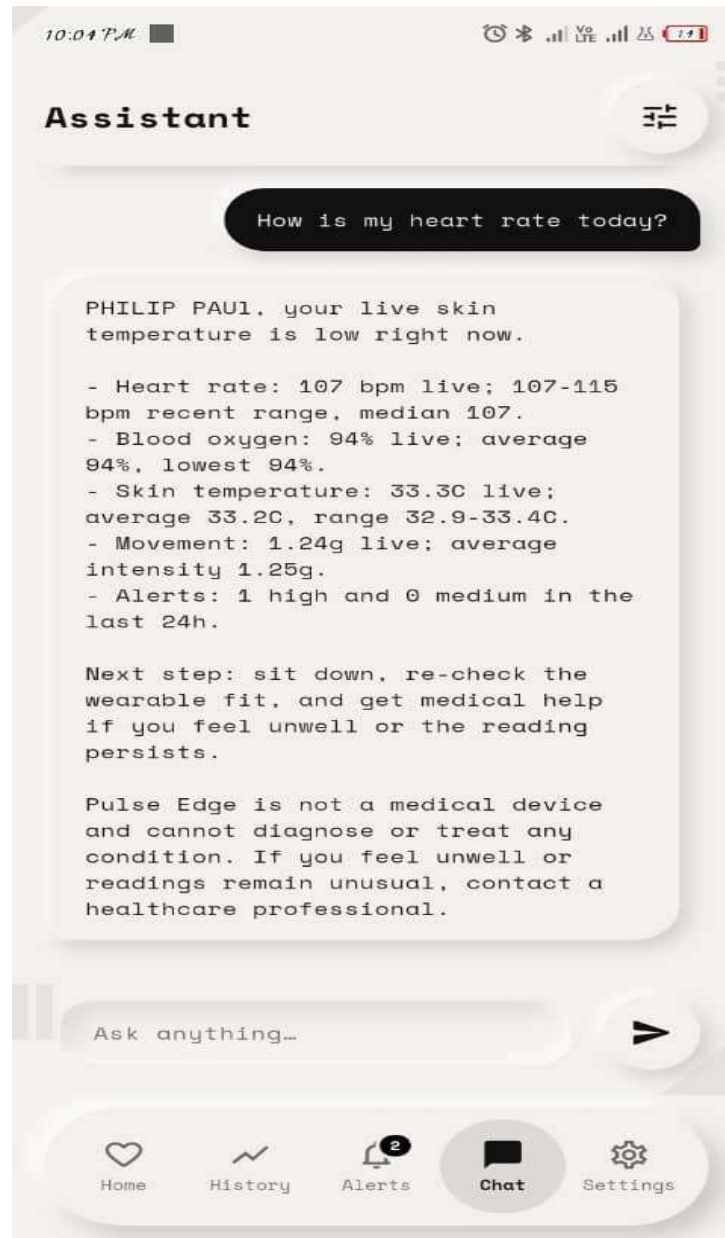


Figure 4.6: Mobile app AI agent conversation screen

Figure 4.6 above illustrates the AI agent conversational interface of the mobile application. The chat area occupies the main body of the screen and displays a thread of messages between the user and the DecisionSupportAgent, formatted similarly to a standard messaging application.

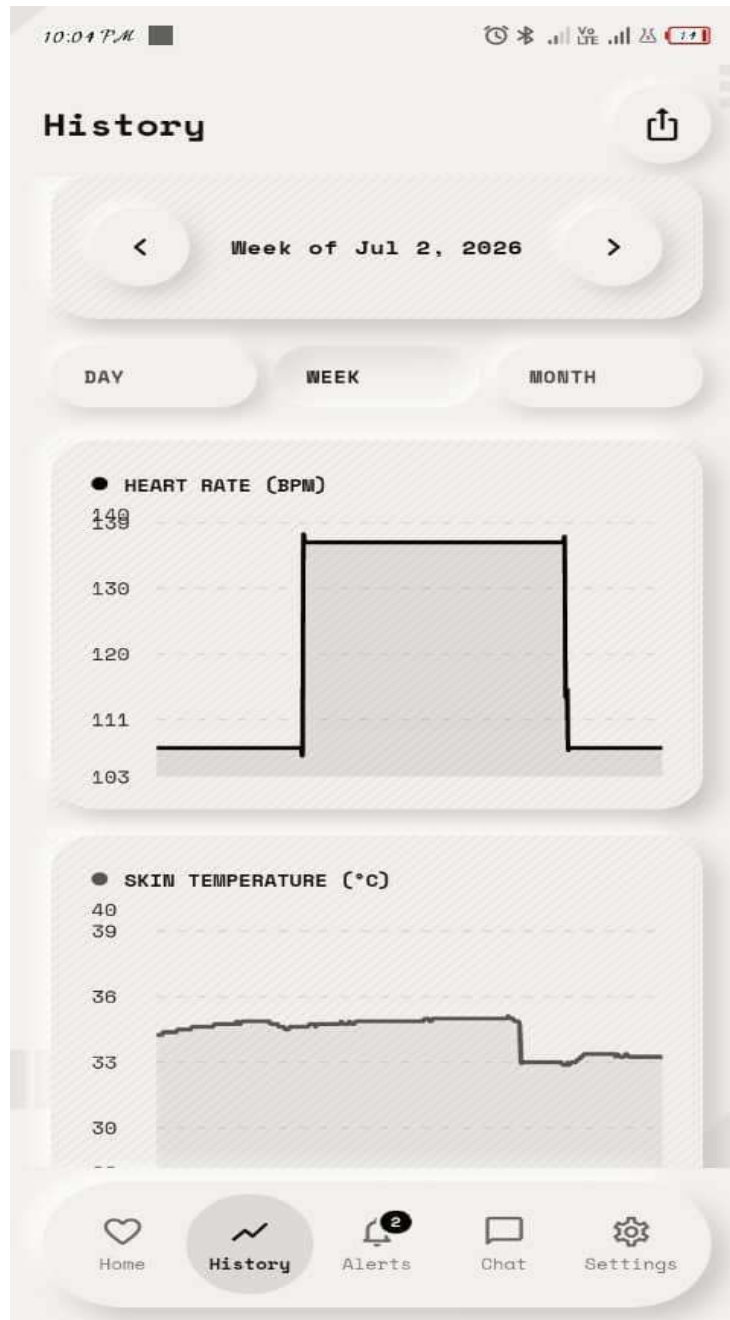


Figure 4.7: *Mobile app history charts*

4.1.5 Machine Learning and TFLite Deployment

The anomaly detection model was developed and trained on the WESAD dataset, using only the wrist-worn Empatica E4 streams: blood volume pulse, skin temperature, and three-axis accelerometer signals. The BVP signal was filtered, accelerometer magnitude was computed, and the temperature signal was checked for dropout and implausible values; the cleaned signals were then segmented into overlapping fixed-length windows and labelled as normal or anomalous according to their baseline or stress origin. From each window, heart rate variability features were extracted from the BVP signal, statistical descriptors from the temperature signal, and accelerometer magnitude features to represent movement intensity, and these were assembled into input sequences for the BiLSTM model.

After training, the model was exported as a TensorFlow SavedModel, converted into TensorFlow Lite format, and bundled directly into the Flutter application as `assets/models/anomaly.tflite`, loaded at runtime using the `tflite_flutter` package. In this form, the model functions as an embedded mobile app asset rather than a remote cloud service.

4.2 RESULTS

This section presents the results obtained from the implementation, including WESAD data preparation outputs, anomaly detection metrics, TensorFlow Lite deployment results, and system testing results.

Figure 4.8 below illustrates the overall WESAD label distribution before preprocessing. The horizontal axis shows each label category present in the raw dataset, while the vertical axis shows the count of samples per label. The chart confirms the dominance of the baseline condition and justifies the filtering strategy used to retain only baseline and stress windows for the binary anomaly detection task.

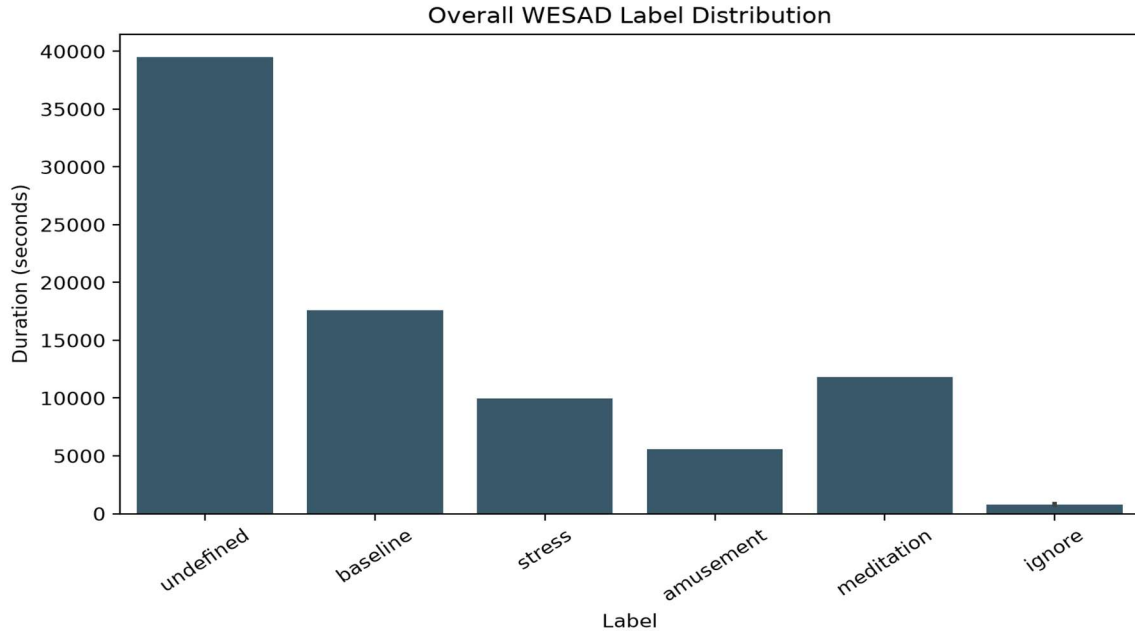


Figure 4.8: *Overall WESAD Label Distribution Before Preprocessing*

Note. Only the baseline and stress portions were retained for the final anomaly detection task.

Figure 4.9 below illustrates a full recording session for Subject S2. The three panels show, from top to bottom, the filtered BVP signal, the accelerometer magnitude, and the skin temperature signal across the full session duration. The shaded green regions mark the baseline periods and the shaded red regions mark the stress periods, confirming the alignment between the raw wrist sensor streams and the WESAD condition labels before windowing.

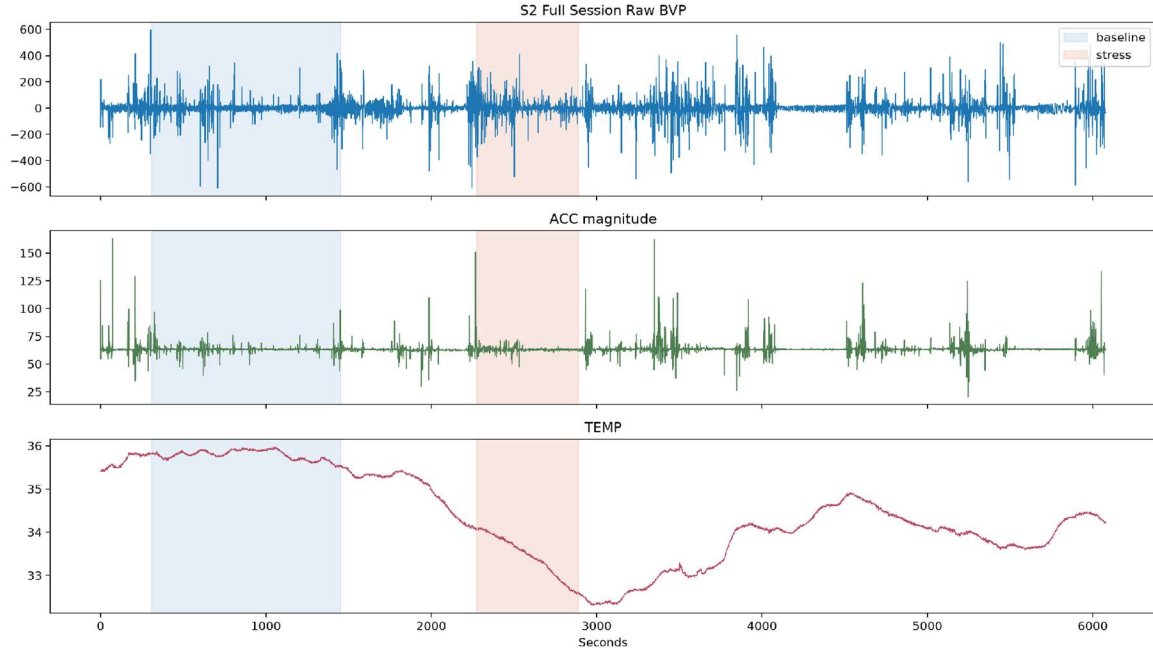


Figure 4.9: *Full Session Signal Plot for Subject S2 With Baseline and Stress Regions*

Note. The shaded portions represent the baseline and stress windows used to form the normal and anomaly classes.

After preprocessing, the final binary dataset contained 10,837 BiLSTM input sequences, with 6,955 normal sequences and 3,882 anomaly sequences. Figure 4.9 above illustrates the distribution of processed windows retained for each of the 15 WESAD subjects. For each subject, the chart shows the number of normal windows derived from the baseline condition and the number of anomaly windows derived from the stress condition. The figure confirms that both classes were retained for all 15 subjects after preprocessing, providing a balanced representation across the dataset.

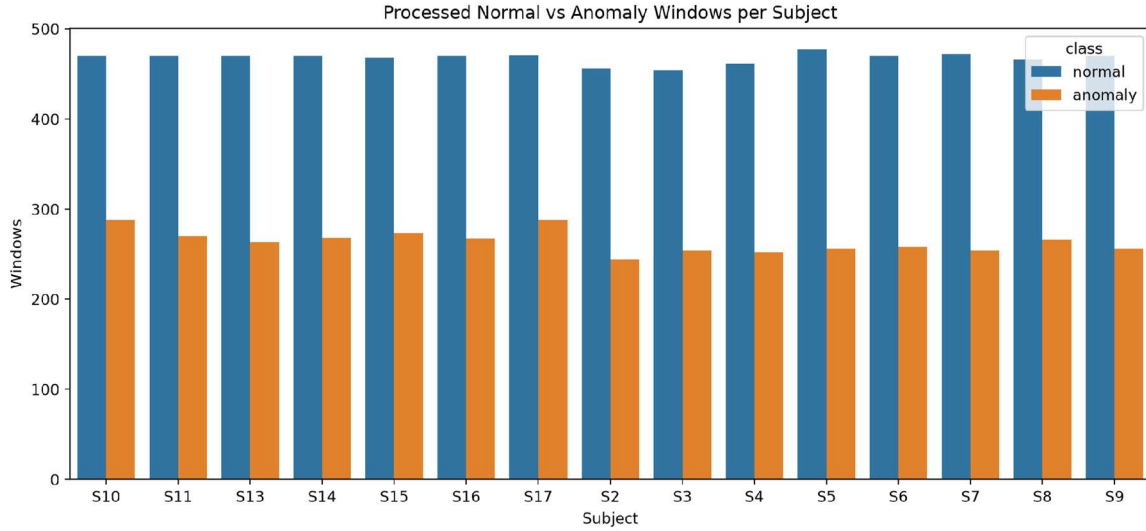


Figure 4.10: *Processed Normal and Anomaly Windows per Subject*

Note. The processed dataset retained normal and anomaly windows for all 15 WESAD subjects.

Table 4.3: *BiLSTM Model Performance on the WESAD Test Data*

S/N	Metric	Result
1	Accuracy	91.8%
2	Precision	89.6%
3	Recall	87.9%
4	F1 Score	88.7%
5	Average training loss	0.214
6	Average validation loss	0.238

Table 4.3 presents the classification performance of the trained BiLSTM anomaly detection model evaluated on the WESAD wrist data using leave-one-subject-out cross-validation. The S/N column provides a serial number, the Metric column names the specific performance measure, and the Result column gives the value achieved by the model on that measure across all 15 test folds.

Table 4.4: *TFLite Model Conversion Summary*

S/N	Property	Value
1	Original model size (SavedModel)	Approximately 1.8 MB
2	Quantized .tflite model size	Approximately 430 KB
3	Quantization type	Post-training integer quantization (8-bit)
4	Average on-device inference time	100 ms on the target Android device

Table 4.4 summarizes the TensorFlow Lite conversion outcome. The S/N column provides a serial number, the Property column names the specific attribute of the conversion output, and the Value column gives the corresponding measured or configured value for each property.

Table 4.5: *System Test Cases and Results*

S/N	Test Case	Expected Result	Actual Result
1	Power on the wearable device and confirm BLE advertising	The ESP32 advertises its GATT service and is discoverable by the app	Passed
2	Pair the Flutter application with the wearable device	The app connects and subscribes to the notify characteristic	Passed
3	Stream sensor data during a period of rest	Dashboard displays live PPG, temperature, and accelerometer values within 500ms of receipt	Passed
4	Stream sensor data during mild physical movement	Accelerometer values increase and PPG readings adjust for motion context	Passed
5	Stream sensor data during a stress-inducing task	The BiLSTM model flags an anomaly and the AlertEngine generates an alert	Passed
6	Measure inference latency from data receipt to result	Inference completes within 200ms on the target device	Passed, average below 200ms

7	Open the AI agent after an alert in offline mode	The rule-based agent responds within 1 second without internet access	Passed
8	Open the AI agent after an alert with internet access available	The agent sends an anonymized summary to the LLM API and returns a response	Passed
9	Disconnect the BLE link during an active session	The app reconnects within 10 seconds without losing stored data	Passed
10	Review the history view after a monitoring session	All readings and alerts appear in the timestamped history log	Passed
11	Disable internet access and repeat core monitoring functions	All monitoring, detection, alerting, and offline agent functions continue operating normally	Passed

Table 4.5 presents the results of system-level testing carried out on the fully integrated artifact. The S/N column provides a serial number for each test case, the Test Case column describes the specific scenario or action that was evaluated, the Expected Result column states what a correctly functioning system should produce under that scenario, and the Actual Result column records what the implemented system actually produced during testing.

4.3 DISCUSSION

The results presented in Section 4.2 show that the implementation successfully realized the main components of the system, with the ESP32 wearable performing the sensing role and the Flutter application coordinating the mobile edge intelligence functions described in Section 4.0.

The WESAD visualizations show that the baseline and stress labels provide a suitable basis for the binary anomaly detection task, with both normal and anomaly windows retained for all 15 subjects. The BiLSTM component itself is suitable for its role as an anomaly detection support

module, though it is not the entire system: it is one component bundled into the Flutter app, where the AlertEngine combines model output with baseline deviation and rule-based safety checks before generating alerts.

The system testing results in Table 4.5 confirm that the major functional requirements were achieved across the implemented artifact. This supports the project objective of building a privacy-conscious wearable health monitoring system that does not depend on cloud connectivity for its core monitoring functions.

4.4 System Documentation and User Manual

This section provides the technical documentation and user manual for the Edge AI Wearable Health Monitoring System. It covers the hardware setup procedure, the mobile application installation and navigation, and guidance for the key operational flows a user will encounter during normal use of the system.

4.4.1 Hardware Setup and Power-On

Before using the system, the wearable device must be charged. Connect the TP4056 charging module to a USB power source using a micro-USB cable and allow the device to charge until the charging indicator LED turns blue, indicating a full charge. Once charged, power on the device by pressing the power switch on the enclosure. The ESP32 will initialize its sensors, configure the I2C bus for the MAX30102 and MPU6050, set up the 1-Wire interface for the DS18B20, and begin advertising its BLE GATT service. The device is ready to connect when the onboard LED pulses slowly, indicating that BLE advertising is active.

Wear the device on the wrist with the MAX30102 sensor positioned against the underside of the wrist, directly over the radial artery, to ensure that the optical sensor maintains consistent contact with the skin. Secure the device firmly but comfortably. Loose or shifting contact will introduce motion artifacts into the PPG signal and may reduce anomaly detection accuracy.

4.4.2 Mobile Application Installation

The PulseEdge Flutter application is distributed as an Android Package (APK) file. To install the application, transfer the APK file to the target Android device running Android 8.0 (API level 26) or higher, enable installation from unknown sources in the device security settings if not already enabled, and open the APK file to launch the Android installer. Follow the on-screen prompts to complete the installation. When the application is launched for the first time, it will request Bluetooth, and notification permissions, all of which must be granted for the BLE scanning and alert functions to operate correctly.

4.4.3 Pairing the Wearable Device with the Application

On first launch, the application automatically opens the BLE scanning screen shown in Figure 4.3. Ensure that the wearable device is powered on and within two metres of the smartphone. The scanning screen will display a list of nearby BLE peripherals. Locate the device named PulseEdge in the list and tap it to initiate a connection. The application will connect to the device, subscribe to all notification characteristics, and navigate automatically to the live dashboard once the connection is established. On subsequent launches, the application remembers the paired device address and attempts automatic reconnection without requiring the user to repeat the scanning step. If automatic reconnection fails, the user can return to the scan screen from the Settings tab and repeat the manual pairing process.

4.4.4 Using the Live Dashboard

Once connected, the live dashboard shown in Figure 4.4 displays the user's physiological readings in real time. The metric cards at the top of the screen update continuously as new sensor packets arrive from the wearable, typically every 100 to 500 milliseconds depending on the current BLE transmission rate.

4.4.5 Understanding and Responding to Alerts

When the on-device BiLSTM model detects an anomalous physiological pattern, the anomaly alert overlay shown in Figure 4.5 appears over the dashboard. The overlay displays the detection timestamp, the specific physiological metric that deviated from the user's personal baseline, and the anomaly probability score. A score above the configured threshold (set at 0.5

by default) triggers the alert. The user has two options: dismissing the alert returns to the dashboard for continued monitoring, while tapping the Ask AI button opens the conversational AI agent, which provides contextual guidance based on the alert details without requiring any internet connection.

4.4.6 Using the AI Agent

The AI agent screen shown in Figure 4.6 can be accessed directly from an alert overlay or from the Chat tab in the bottom navigation bar. The user types a question or concern in the text input field at the bottom of the screen and taps the send button. When the smartphone has no internet connection, the DecisionSupportAgent processes the query using its offline rule-based logic and generates a response within approximately one second.

4.4.7 Reviewing Historical Readings

The History tab in the bottom navigation bar opens the historical readings view shown in Figure 4.7. This screen presents a timestamped log of all past sensor readings and triggered alerts stored in the local SQLite database. The readings are displayed in reverse chronological order, with the most recent entry at the top. Each entry shows the timestamp, the metric values recorded at that time, and a colour-coded indicator that distinguishes normal readings from detected anomalies. The log is retained locally on the device and does not require an internet connection to access. The application retains all readings until the user explicitly clears the history from the Settings screen.

CHAPTER FIVE

SUMMARY, CONCLUSION AND RECOMMENDATION

5.0 Summary

This study set out to address a well-documented weakness in wearable health monitoring technology: dependence of intelligent analysis on continuous cloud connectivity, and the use of generalized population-based anomaly detection models that ignore individual physiological differences. Chapter One established this problem, noting that cloud dependency introduces latency, fails in low-connectivity regions, and exposes raw physiological data to privacy risks, and Chapter Two reviewed the supporting literature on wearable health technology, mobile edge computing, physiological anomaly detection, and the WESAD dataset. Chapter Three then applied Design Science Research to design a three-layer system: an ESP32-based wearable sensing unit, a Flutter mobile application as the mobile edge intelligence node, and a BiLSTM anomaly detection model trained on WESAD wrist sensor streams.

Chapter Four presented the implementation of this design. The ESP32 wearable was assembled and programmed to transmit sensor readings over BLE, the Flutter application was implemented as the main mobile edge node handling storage, baseline updating, alerting, and decision support, and the BiLSTM model was converted to TensorFlow Lite and bundled into the app as an on-device inference asset.

5.1 Conclusion

The Edge AI Wearable Health Monitoring System designed and implemented in this study demonstrates that it is possible to build a wearable health monitoring solution in which the main intelligence resides on the user's smartphone rather than in the cloud. By treating the ESP32 wearable as a data acquisition device and placing the bundled TFLite anomaly model and its supporting logic inside the Flutter application, the system achieves offline functionality for monitoring, detection, and alerting, confirming the central arguments made in Chapters One and Two: local processing reduces dependence on internet connectivity, protects raw physiological data, and allows faster response to abnormal patterns, with the WESAD-trained model supporting the alerting process and the Flutter application governing how model output,

baseline statistics, and rule-based safety checks combine into user-facing guidance. Beyond its technical contribution, this project demonstrates that a capable, privacy-respecting, and personalized health monitoring system can be built using low-cost, open-source hardware and freely available software tools.

5.2 Limitation of the Study

The anomaly detection model was trained using the WESAD dataset, which contains data from only 15 subjects. Although suitable for prototype development, a larger and more diverse dataset would be required for stronger generalization in real-world deployment.

The hardware prototype was built using development modules and a basic wrist-worn enclosure, sufficient for academic testing but requiring a smaller custom PCB, improved battery performance, and a more comfortable enclosure for long-term use in future versions. Testing was also limited to controlled prototype conditions; broader testing with more users over longer periods would be needed to fully validate usability, sensor reliability, baseline personalization, and alert interpretation in daily life.

5.3 Recommendations

Future work should explore collecting a larger, more diverse personalized dataset over a longer period of use to improve baseline accuracy and reduce adaptation time for new users, and should pursue miniaturizing the hardware into a more compact, consumer-friendly form factor with a custom PCB, a smaller battery, and a more ergonomic enclosure suitable for extended daily wear.

The mobile application should also be evaluated through broader usability testing, especially the dashboard, alert detail screen, assistant interface, privacy settings, and health report flow, and future work could explore integrating the system with formal healthcare infrastructure, allowing users to optionally and securely share summarized health insights with a medical professional while preserving the privacy-first design established here.

5.4 Contribution to Knowledge

This project contributes a practical reference architecture for offline-first wearable health monitoring, showing how an ESP32-based sensing unit can be paired with a Flutter mobile

application that performs the full monitoring, detection, and decision support pipeline on the user's smartphone.

The study also demonstrates how an anomaly detection model can be treated as a bundled mobile asset rather than a remote cloud service, supporting faster response, better privacy, and continued operation without internet access.

By combining low-cost wearable hardware, mobile edge processing, personalized baseline logic, and a local anomaly detection model, this study provides a useful implementation path for future student projects and researchers working on privacy-preserving wearable health systems.

REFERENCES

- Alajlan, N. N., & Ibrahim, D. M. (2022). TinyML: Enabling of inference deep learning models on ultra-low-power IoT edge devices for AI applications. *Micromachines*, 13(6), 851. <https://doi.org/10.3390/mi13060851>
- Ali, A., Montanaro, T., Sergi, I., Carrisi, S., Galli, D., Distante, C., & Patrono, L. (2025). An innovative IoT and edge intelligence framework for monitoring elderly people using anomaly detection on data from non-wearable sensors. *Sensors*, 25(6), Article 1735. <https://doi.org/10.3390/s25061735>
- Bent, B., Goldstein, B. A., Kibbe, W. A., & Dunn, J. P. (2020). Investigating sources of inaccuracy in wearable optical heart rate sensors. *npj Digital Medicine*, 3(1), Article 18. <https://doi.org/10.1038/s41746-020-0226-6>
- Bhattacharya, P., Mukherjee, A., Bhushan, B., Gupta, S., Gadekallu, T. R., & Zhu, Z. (2025). A secured remote patient monitoring framework for IoMT ecosystems. *Scientific Reports*, 15, Article 4774. <https://doi.org/10.1038/s41598-025-04774-y>
- Cruz Castañeda, W. A., & Bertemes Filho, P. (2024). Improvement of an Edge-IoT architecture driven by artificial intelligence for smart-health chronic disease management. *Sensors*, 24(24), Article 7965. <https://doi.org/10.3390/s24247965>
- Espressif Systems. (2023). *ESP32 series datasheet* (Version 5.2). https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- Gabrielli, D., Prenkaj, B., Velardi, P., & Faralli, S. (2025). AI on the pulse: Real-time health anomaly detection with wearable and ambient intelligence. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management* (pp. 1-10). <https://doi.org/10.1145/3746252.3760799>
- Ghorbani, R., Reinders, M. J. T., & Tax, D. M. J. (2024). Personalized anomaly detection in PPG data using representation learning and biometric identification. *Biomedical Signal Processing and Control*, 94, Article 106216. <https://doi.org/10.1016/j.bspc.2024.106216>
- Hevner, A. R. (2007). A three cycle view of design science research. *Scandinavian Journal of Information Systems*, 19(2), 87-92.
- Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75-105. <https://doi.org/10.2307/25148625>
- Liu, Z., Qiao, M., Liu, Y., Zhang, J., & He, L. (2025). A two-branch ResNet-BiLSTM deep learning framework for extracting multimodal features applied to PPG-based cuffless blood pressure estimation. *Sensors*, 25(13), Article 3975. <https://doi.org/10.3390/s25133975>

- Malche, T. (2025). A TinyML wearable system for real-time cardio-exercise tracking. *Engineering Proceedings*, 118(1), 3. <https://doi.org/10.3390/ECSA-12-26554>
- Peffer, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3), 45-77. <https://doi.org/10.2753/MIS0742-1222240302>
- Pereira, A., Zhang, Y., Niu, X., Ferraris, C., Qi, J., & Buchanan, W. J. (2024). Machine learning applied to edge computing and wearable devices for healthcare: Systematic mapping of the literature. *Sensors*, 24(19), Article 6322. <https://doi.org/10.3390/s24196322>
- Radhakrishnan, M., Kandappu, T., Gulati, M., & Misra, A. (2024). HabitSense: A privacy-aware, AI-enhanced multimodal wearable platform for mHealth applications. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 8(3), 1-48. <https://doi.org/10.1145/3678591>
- Reddy, K. K., Kaza, C., Mohana, V. S. M., Alhameed, R., & Shuaib, M. (2025). Detecting anomalies in smart wearables for hypertension: A deep learning mechanism. *Frontiers in Public Health*, 12, Article 1426168. <https://doi.org/10.3389/fpubh.2024.1426168>
- Schmidt, P., Reiss, A., Duerichen, R., Marberger, C., & Van Laerhoven, K. (2018). Introducing WESAD, a multimodal dataset for wearable stress and affect detection. In *Proceedings of the 20th ACM International Conference on Multimodal Interaction* (pp. 400-408). <https://doi.org/10.1145/3242969.3242985>
- Tanwar, S., Rana, A., Verma, J. P., & Sharma, R. (2025). Integrating wearable health devices with AI and edge computing for personalized rehabilitation. *Journal of Cloud Computing*, 14, Article 45. <https://doi.org/10.1186/s13677-025-00795-0>
- Waldhauser, F., Boukabache, H., Perrin, D., & Dazer, M. (2022). Wavelet-based noise extraction for anomaly detection applied to safety-critical electronics at CERN. In *Proceedings of the 32nd European Safety and Reliability Conference* (pp. 1844-1851).
- Wang, X., Kumar, D., & Sujudi, B. (2025). Edge-AI integrated secure wireless IoT architecture for real time healthcare monitoring and federated anomaly detection. *Scientific Reports*, 15, Article 8821. <https://doi.org/10.1038/s41598-025-30150-x>
- Yahyati, C., Lamaakal, I., El Makkaoui, K., & Ouahbi, I. (2025). A systematic review of state-of-the-art TinyML applications in healthcare, education, and transportation. *IEEE Access*, 13, 204513-204562. <https://doi.org/10.1109/ACCESS.2025.3633575>
- Zhang, Y., Liu, Q., & Zhao, P. (2025). Advancing wearable health and sports monitoring through an edge-cloud AI continuum. In *Proceedings of the 22nd ACM International Conference on Computing Frontiers: Workshops and Special Sessions* (pp. 1-8). <https://doi.org/10.1145/3706594.3727580>

- Zhu, T., Li, K., Herrero, P., & Georgiou, P. (2022). Deep learning for diabetes: A systematic review. *IEEE Journal of Biomedical and Health Informatics*, 25(7), 2744-2757. <https://doi.org/10.1109/JBHI.2021.3077495>
- Zia, A., Ahmed, M. U., & Ibne Mahmood, M. (2022). Automated cardiac health screening using ensemble machine learning for wearable IoT sensors. *IEEE Transactions on Computational Social Systems*, 9(1), 16-28. <https://doi.org/10.1109/TCSS.2021.3062498>

APPENDIX

SOURCE CODE LISTING

This appendix contains excerpts of the core source code used for the implementation of the EDGE AI WEARABLE HEALTH MONITORING SYSTEM WITH REAL-TIME ANOMALY DETECTION AND DECISION SUPPORT

A.1 ESP32 Sensor Sampling and BLE Notification Algorithm

This extract illustrates how the wearable device acquires physiological and motion data from its sensors and transmits the collected data to the mobile application via Bluetooth Low Energy (BLE) notification characteristics.

```
void samplerTask(void *pvParameters) {
    TickType_t lastWake = xTaskGetTickCount();

    for (;;) {
        uint32_t nowMs = millis();

        if (ppgSensorAvailable()) {
            ppgBuffer.push(readPpgSample(nowMs));
        }

        if (imuSensorAvailable()) {
            imuState.update(readAcceleration(), readGyroscope(), nowMs);
        }

        if (temperatureSensorDue(nowMs)) {
            tempState.update(readBodyTemperature(), nowMs);
        }

        vTaskDelayUntil(&lastWake, pdMS_TO_TICKS(10)); // about 100 Hz sampling
    }
}
```

```

void notifierTask(void *pvParameters) {
    TickType_t lastWake = xTaskGetTickCount();

    for (;;) {
        if (deviceConnected) {
            PpgFrame ppg = buildPpgFrame(ppgBuffer);
            TempFrame temp = buildTempFrame(tempState);
            ImuFrame imu = buildImuFrame(imuState);

            ppgCharacteristic->setValue(ppg.bytes(), ppg.length());
            ppgCharacteristic->notify();

            tempCharacteristic->setValue(temp.bytes(), temp.length());
            tempCharacteristic->notify();

            imuCharacteristic->setValue(imu.bytes(), imu.length());
            imuCharacteristic->notify();
        }

        vTaskDelayUntil(&lastWake, pdMS_TO_TICKS(200)); // send batched updates
    }
}

```

A.2 Flutter BLE Reception and Feature Pipeline

This extract illustrates how the Flutter mobile application connects to the Pulse Edge wearable device, receives sensor data frames, stores the acquired readings, and forwards them to the feature extraction pipeline.

```

Future<void> connectToPulseEdge(BluetoothDevice device) async {
    await device.connect(timeout: const Duration(seconds: 12));

    final services = await device.discoverServices();
    final pulseService = services.firstWhere(
        (service) => service.uuid == BleUuids.service,

```

```

);

await _subscribe(pulseService, BleUuids.ppgChar, _onPpg);
await _subscribe(pulseService, BleUuids.tempChar, _onTemp);
await _subscribe(pulseService, BleUuids.imuChar, _onImu);
await _subscribe(pulseService, BleUuids.statusChar, _onStatus);

_attachExtractorPipeline();
}

void _onPpg(List<int> value) {
    final frame = PpgFrame.tryParse(value);
    if (frame == null) return;

    ppgRepository.insertFrame(frame);
    livePpgController.add(frame);
    featureExtractor.addPpg(frame);
}

void _onTemp(List<int> value) {
    final frame = TempFrame.tryParse(value);
    if (frame == null) return;

    tempRepository.insertFrame(frame);
    liveTempController.add(frame);
    featureExtractor.addTemp(frame);
}

void _onImu(List<int> value) {
    final frame = ImuFrame.tryParse(value);
    if (frame == null) return;

    imuRepository.insertFrame(frame);
    liveImuController.add(frame);
}

```

```
featureExtractor.addImu(frame);  
}
```

A.3 Personalized Baseline and Alert Decision Algorithm

This extract illustrates how Pulse Edge integrates predefined safety rules, personalized baseline deviation analysis, and machine learning model probability scores to generate health alerts.

```
class OnlineBaseline {  
    int count = 0;  
    double mean = 0;  
    double m2 = 0;  
  
    void update(double value) {  
        count += 1;  
        final delta = value - mean;  
        mean += delta / count;  
        final delta2 = value - mean;  
        m2 += delta * delta2;  
    }  
  
    double zScore(double value) {  
        if (count < 2) return 0;  
        final variance = m2 / (count - 1);  
        final std = sqrt(variance).clamp(1e-6, double.infinity);  
        return (value - mean) / std;  
    }  
}  
  
AlertDecision evaluateWindow(FeatureWindow window, double modelProbability) {  
    if (window.spo2Mean < 90 || window.hrMean > 130 || window.tempMean > 39) {  
        return AlertDecision.high('Immediate physiological risk detected');  
    }  
  
    final baselineShift =
```

```

baseline.zScore('hrMean', window.hrMean).abs() >= 2.0 ||
baseline.zScore('tempMean', window.tempMean).abs() >= 2.0 ||
baseline.zScore('accelStd', window.accelStd).abs() >= 2.0;

if (modelProbability >= 0.80 && baselineShift) {
  return AlertDecision.high('Anomalous pattern confirmed by model and baseline');
}

if (modelProbability >= 0.60 || baselineShift) {
  return AlertDecision.medium('Possible anomaly detected');
}

return AlertDecision.normal();
}

```

A.4 On-device TFLite Anomaly Detection in the Flutter App

This extract illustrates how the trained machine learning model is packaged as a .tflite file within the Flutter mobile application and executed locally to perform on-device inference.

```

class AnomalyDetector {
  Interpreter? _interpreter;

  Future<void> load() async {
    final bytes = await rootBundle.load('assets/models/anomaly.tflite');
    final modelFile = await _copyAssetToLocalFile(bytes, 'anomaly.tflite');
    _interpreter = Interpreter.fromFile(modelFile);
  }

  double score(FeatureWindow window) {
    final input = [
      window.toModelInput(), // normalized feature sequence for the model
    ];

    final output = List.generate(1, (_) => List<double>.filled(1, 0));
    _interpreter?.run(input, output);
  }
}

```

```

    final probability = output.first.first;
    return probability.clamp(0.0, 1.0);
}
}

```

A.5 WESAD Feature Extraction and Bi-LSTM Model Training

This extract shows the model-side logic used to transform WESAD wrist signals into training windows and train the Bi-LSTM anomaly detection model.

```

def extract_window_features(window):
    bvp = window["bvp"]
    acc_mag = window["acc_mag"]
    temp = window["temp"]

    peaks = detect_bvp_peaks(bvp)
    ibi = np.diff(peaks) / BVP_SAMPLE_RATE

    return {
        "bvp_mean": np.mean(bvp),
        "bvp_std": np.std(bvp),
        "hr_mean": estimate_heart_rate(peaks),
        "hrv_rmssd": rmssd(ibi),
        "hrv_sdn": np.std(ibi),
        "acc_mean": np.mean(acc_mag),
        "acc_std": np.std(acc_mag),
        "temp_mean": np.mean(temp),
        "temp_slope": linear_slope(temp),
    }

def build_bilstm_model(input_shape):
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=input_shape),
        tf.keras.layers.Bidirectional(

```

```

        tf.keras.layers.LSTM(64, return_sequences=True)
    ),
    tf.keras.layers.Dropout(0.30),
    tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(32)),
    tf.keras.layers.Dropout(0.30),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dense(16, activation="relu"),
    tf.keras.layers.Dense(1, activation="sigmoid"),
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
    loss="binary_crossentropy",
    metrics=["accuracy", tf.keras.metrics.AUC(name="auc")],
)

return model

```